# Kapitel 6

## Software Testing

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

# Agenda

## Abschnitt 6.1
Einführung zum Software Testing

Prof. Dr. Philipp Rohde 25WiSe-SWT | 249

## Abschnitt 6.2
Teststufen

Prof. Dr. Philipp Rohde 25WiSe-SWT | 258

## Abschnitt 6.3
Statische Tests

Prof. Dr. Philipp Rohde 25WiSe-SWT | 275

## Abschnitt 6.4
Dynamische Tests

Prof. Dr. Philipp Rohde 25WiSe-SWT | 292

# Klausurrelevante Lernziele

- Fehler, Mangel, Fehlerwirkung & Fehlerzustand sowie die Fehlerursachenkette erläutern können
- Unterschied zwischen konstruktivem & analytischem Qualitätsmanagement erläutern können
- Unterschied zwischen statischen & dynamischen Tests erläutern können
- Fehlernachtest, Regressionstest, Alpha- & Beta-Test erläutern können
- Die vier Teststufen & deren Testziele nennen können
- Drei Merkmale guter Komponententests nennen & das AAA-Pattern erläutern können
- Kontrollflussgraph für gegebenen Code erstellen & zyklomatische Komplexität berechnen können
- Unterschied zwischen Black-Box- & White-Box-Test erläutern können
- Black-Box-Tests: Äquivalenzklassenbildung & Grenzwertanalyse für Beispiel durchführen können
- White-Box-Tests: Anweisungstest, Zweigtest, Bedingungstest, Mehrfachbedingungstest & Grenze-Inneres-Test für ein Beispiel durchführen können sowie die Beziehungen dieser Tests untereinander nennen können
- Nicht klausurrelevant: nicht-funktionale Tests, Point of Control (PoC), Point of Observation (PoO)

# Option: Zertifizierung

- Certified Tester Foundation Level (CTFL)
  - Aktuell: Version 4.0 (Mai 2024)
- Weltweit anerkannte Zertifizierung für professionelles Software Testing
  - Durch: International Software Testing Qualifications Board (ISTQB)
  - Basis für weitergehende Zertifizierungen (Advanced, Expert, Specialist)
  - Mehr als eine Million Zertifizierungen in 130 Ländern (Stand: Mai 2025)
- Prüfung bei unabhängiger Zertifizierungsstelle
  - Multiple Choice, 40 Fragen, 60 (Native) / 75 Minuten (Non-Native), Bestehensquote: 65%, deutsch
  - Gebühr: ca. 240 € (ggf. 50% Studierendenrabatt)
  - Beispielprüfungen verfügbar
- Vorlesung bereitet teilweise auf die Prüfung vor

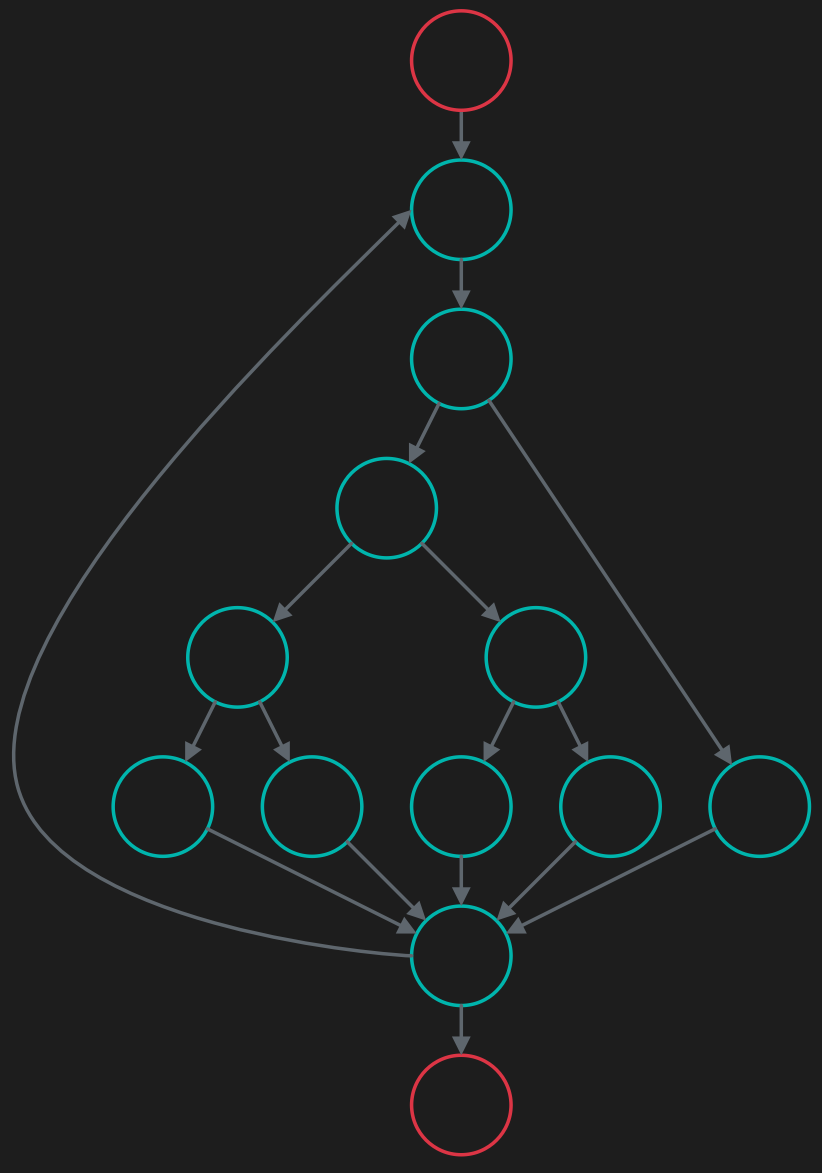
# Abschnitt 6.1

## Einführung zum Software Testing

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

# Vollständiges Testen

- Szenario
  - Ein einfaches Programm besteht aus vier If-Anweisungen & umgebende Schleife, siehe Abbildung
  - Annahme: die Verzweigungen sind voneinander unabhängig und die maximale Anzahl der Schleifendurchläufe kann auf 20 beschränkt werden
- Frage: Wie lange dauert das vollständige Testen aller möglichen Abläufe, wenn 100.000 Tests pro Sekunde möglich sind?

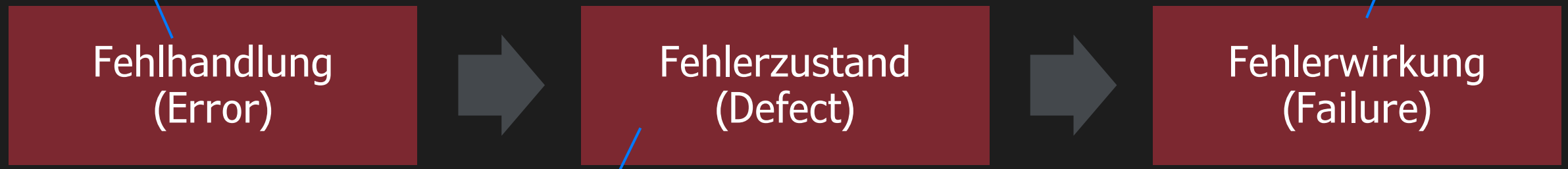


# Fehler, Mangel & Fehlerursachenkette

- Fehler: Nichterfüllung einer festgelegten Anforderung
  - Abweichung zwischen Ist-Verhalten (Test / Betrieb) & Soll-Verhalten (in Spezifikation festgelegt)
  - Verifikation: das System wurde richtig realisiert, es hat keine Fehler
- Mangel: Beeinträchtigung der Verwendbarkeit der Software
  - Wenn berechtigte Erwartung in Bezug auf Gebrauch nicht erfüllt wird
  - Software kann fehlerfrei, aber trotzdem mangelhaft sein
  - Validierung: das richtige System wurde realisiert, es hat keine Mängel

Menschliche Handlung, versehentlich oder absichtlich; Entwickelnde (z.B. fehlerhafte Implementierung) oder Nutzende (z.B. Fehlbedienung)

Äußerer Fehler: wahrnehmbare Abweichung von erwarteter Leistung




Innerer Fehler: Mangel im System, z.B. Bug; kann eine Fehlerwirkung hervorrufen, muss aber nicht

# Sieben Grundsätze des Testens

Testen zeigt die Anwesenheit von Fehlerzuständen, nicht deren Abwesenheit

Vollständiges Testen ist nicht möglich

Frühes Testen spart Zeit & Geld

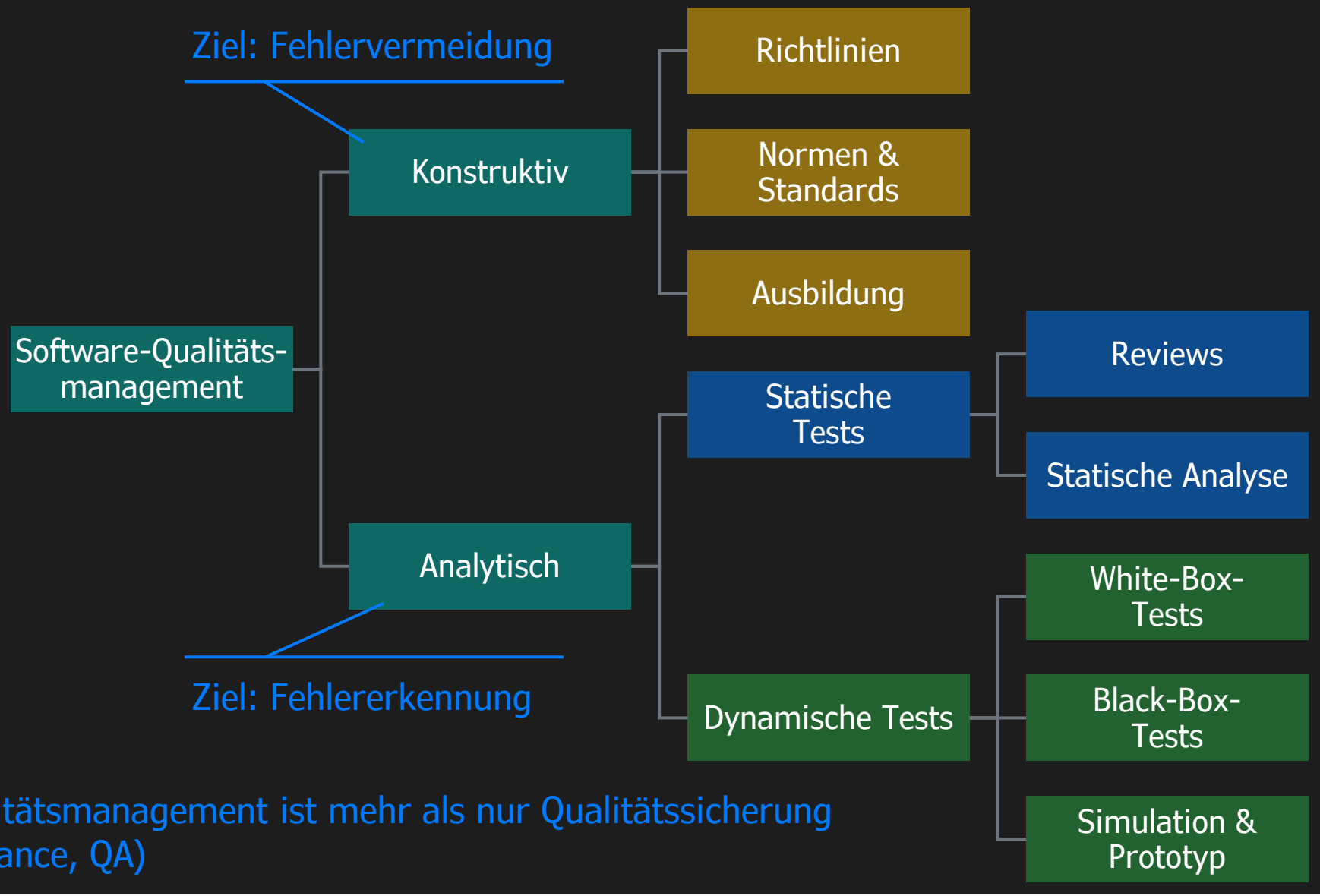
Häufung von Fehlerzuständen (Fehlercluster)

Wiederholungen haben keine Wirksamkeit (Pestizid-Paradoxon)

Testen ist kontextabhängig

Fehlerfrei bedeutet nicht unbedingt auch brauchbar

# Software-Qualitätsmanagement




Beachte: Qualitätsmanagement ist mehr als nur Qualitätssicherung (Quality Assurance, QA)

# Testarten

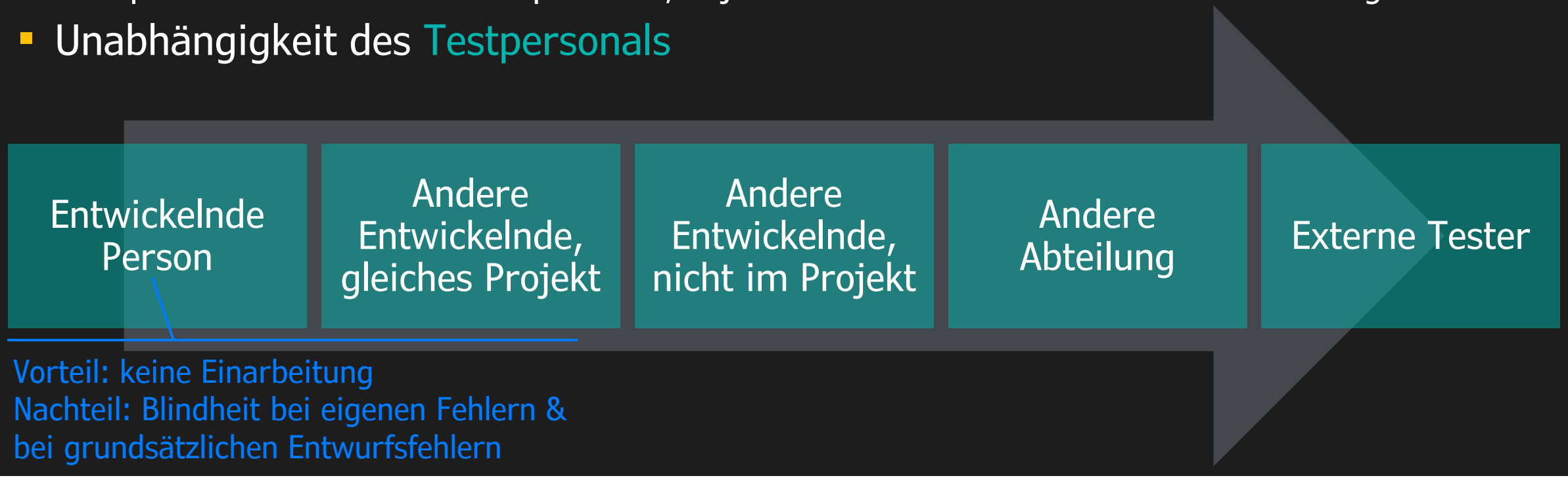
- Statische Tests: System wird beim Test nicht ausgeführt
  - Prüfung von Dokumenten (Anforderungen, UML-Modelle, Quellcode etc.)
  - Findet Fehlerzustände
  - Reviews (z.B. Walkthrough / Inspektion), Code-Analysen, Kontrollflussanalyse, Metriken
- Dynamische Tests: System wird beim Test ausgeführt
  - Prüfung des Verhaltens, insbesondere durch Parameterwerte & Ergebnis
  - Findet Fehlerwirkungen
  - Black-Box-Test: innere Struktur des Codes ist nicht bekannt
  - White-Box-Test: innere Struktur des Codes ist bekannt
- Nicht-funktionale Tests, z.B. Smoke Test, Performanz-, Last-, Stresstest, Benutzbarkeitstest, Zuverlässigkeitstest etc.
- Nach Änderungen
  - Fehlernachtest: wird adressierter Fehlerzustand erfolgreich beseitigt?
  - Regressionstest: werden durch Änderungen neue Fehlerzustände in bereits getesteten Programmteilen erzeugt?

# Testbegriffe (1)

- Testobjekt: Dokument oder Softwaresystem, das geprüft / getestet werden soll
- Testbasis: alle Informationen, die als Basis für Testanalyse & Testentwurf verwendet werden können
- Testfall: Spezifikation von Parametern & Ergebnisse / Verhalten
  - Positivtest: erwartete Ergebnisse bei regulären Parametern / Bedienung (Soll-Verhalten, Normal)
  - Negativtest: Behandlung erwarteter Ausnahmen (Exceptional)
  - Robustheitstest: Behandlung unerwarteter Ausnahmen (Catastrophical)
  - Abstrakte (aka logische) & konkrete Testfälle, z.B. $50 \leq x < 100$ versus $x = 75$
  - Zu Testfällen gehören auch Vor-/Nachbedingungen bzgl. Umgebung
- Testorakel: liefert Soll-Ergebnis
  - Dokumentierte Anforderungen & Spezifikation
  - Prototyp, Back-to-Back-Test (System wird mehrfach entwickelt), Altsysteme (Legacy System) oder Fremdsysteme
- Fehlermaskierung: mehrere Fehler können sich gegenseitig aufheben, die Überlagerung von mehreren Fehlern bleibt eventuell unentdeckt

# Testbegriffe (2)

- Testprozess: Planung, Analyse, Entwurf & Realisierung von Tests; Berichte; Beurteilung der Qualität
- Psychologie des Testens
  - Niemand gibt gerne (mögliche) Fehler zu
  - Wahrnehmung: »Entwicklung ist konstruktiv, Testen ist destruktiv«
  - Reproduzierbarkeit sowie respektvolle, objektive & konstruktive Kommunikation nötig
- Unabhängigkeit des Testpersonals

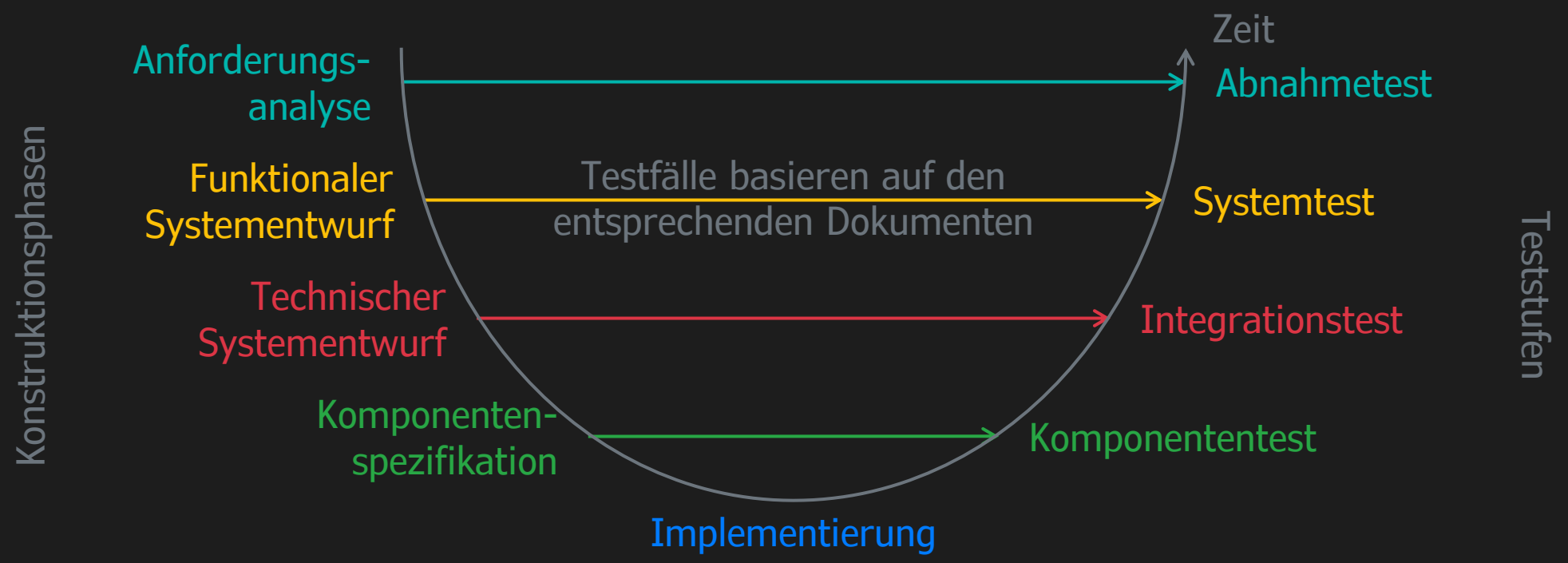


# »How IT people see each other« (2016)

# Abschnitt 6.2

## Teststufen

# Teststufen


Zeit
Anforderungs-
analyse
Abnahmetest
Konstruktionsphasen
Funktionaler
Systementwurf
Testfälle basieren auf den
entsprechenden Dokumenten
Systemtest
Teststufen
Technischer
Systementwurf
Integrationstest
Komponenten-
spezifikation
Komponententest
Implementierung

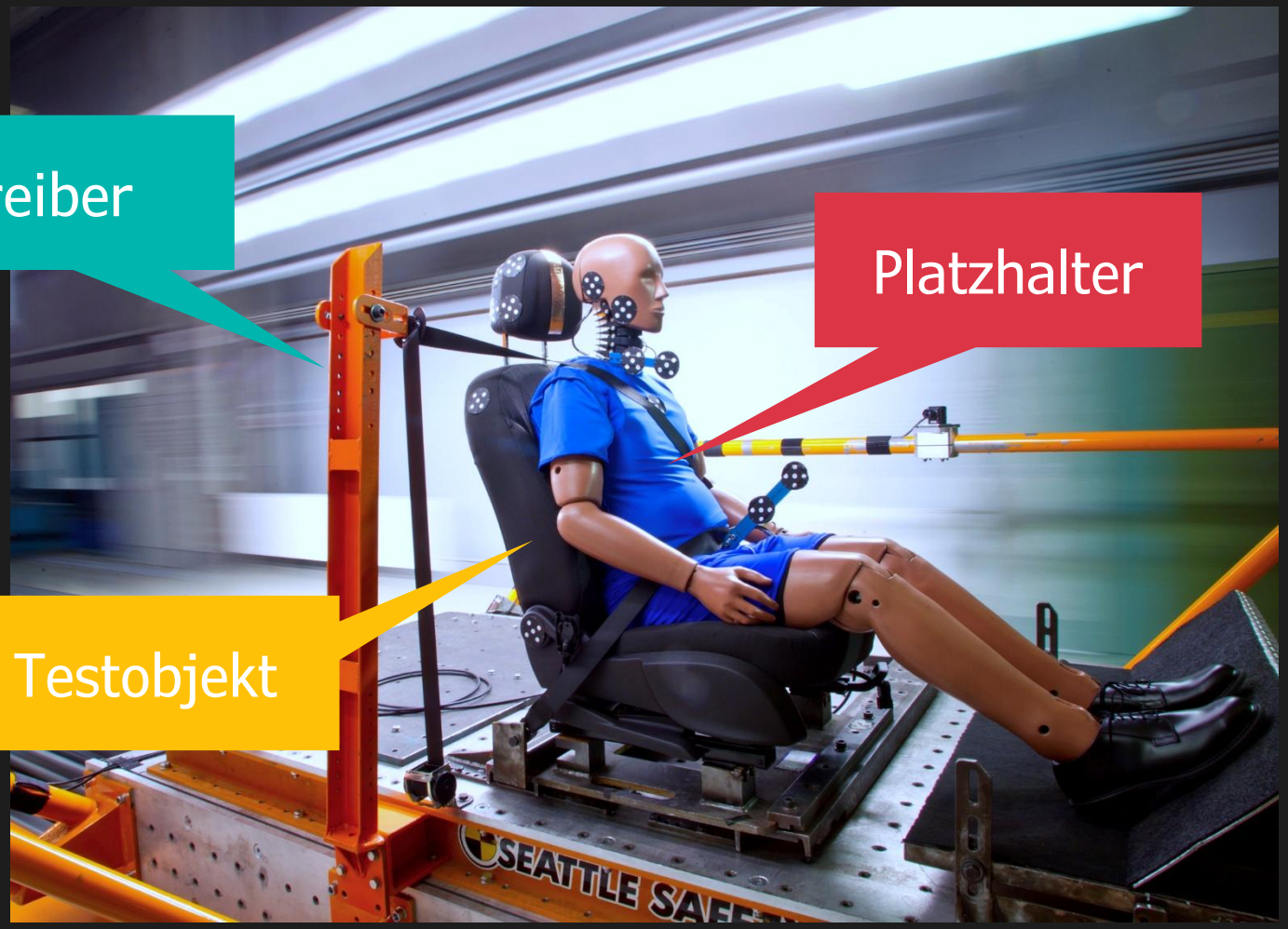
# Teststufen

- Komponententest (Unit Test)
  - Erfüllt jede Komponente für sich die Spezifikation?
- Integrationstest (Integration Test)
  - Arbeiten Komponenten wie im technischen Systementwurf zusammen?
- Systemtest (System Test)
  - Erfüllt das System als Ganzes die Spezifikation?
- Abnahmetest (Acceptance Test)
  - Weist das System die (aus Sicht der Auftraggebenden) vereinbarten Leistungsmerkmale auf? Ist das System einsatztauglich?
- Unterschiede bei den Teststufen
  - Ziele, Testverfahren, Testbasis & Testobjekte
  - Typische Fehlerzustände & Fehlerwirkungen
  - Testumgebung & Testwerkzeuge
  - Verantwortlichkeiten & spezialisiertes Testpersonal

# Komponententest

- Systematischer Test einer Komponente als dynamischer White-Box-Tests (aka Unit Test, Modultest, Entwicklertest)
- Testziele
  - Realisiert Testobjekt die geforderte Funktionalität (Ein-/Ausgabe) korrekt & vollständig?
  - Weitere Tests auf Robustheit, Effizienz & Wartbarkeit
- Testbasis: UML-Modelle, Systementwurf & Quellcode
- Testumgebung: oft innerhalb der IDE
  - Treiber: Aufruf der Dienste des Testobjekts
  - Platzhalter: Simulation der Dienste, die das Testobjekt importiert
- Testfälle
  - Testfälle decken bestimmte Ein-/Ausgabe-Kombination ab
  - Jede Komponente isoliert von anderen & ohne externe Einflüsse
  - Test-Driven Development: Testfälle als Spezifikation der Software
- Automatisierung wesentlich für Continuous Integration

# Komponententest: Testumgebung



# Mocking (1)

- Problem beim Komponententest
  - Komponente soll isoliert getestet werden, aber das ist nicht immer möglich, weil Umgebung (noch) nicht vorhanden, deren Aufbau zu komplex / zu teuer oder Einsatz von Echtdaten unerwünscht ist
  - Frühzeitiges Testen von Komponenten, auch wenn die Komponenten der importierten Schnittstellen noch nicht implementiert sind
  - »Teste die Funktionalität der Komponente und nicht die Funktionalität der Abhängigkeiten«
- Mocking als Platzhalter für importierten Schnittstellen bei Komponententests
  - Implementierung der Umgebung für die zu testende Komponente, z.B. als »Attrappe« mit Pseudodaten
  - Einsatz erhöht indirekt auch Modularität & Kapselung des Systems

# Mocking (2)

- Dummy: Objekt, das im Code weitergereicht, aber nicht verwendet wird, z.B. um Parameter mit (irrelevanten) Werten zu übergeben
- Fake: Objekt mit (eingeschränkter) Implementierung, die untauglich für den Produktionseinsatz ist, z.B. eine temporär In-Memory-Datenbank
- Stub & Mock: Objekt, welches bei Operationsaufruf entweder unabhängig von den Parametern immer die gleiche (Stub) oder regelbasierte Rückgabe (Mock) liefert, z.B. als temporärer Ersatz für eigene Komponenten, die noch nicht fertig sind
- Spy: Objekt, welches Aufrufe und übergebene Werte protokolliert, z.B. durch Erweiterung von Fake-, Stub- oder Mock-Objekten
- Shim: Abfangen und Behandeln von Funktionsaufrufen an beliebige Komponenten zur Laufzeit, weil der zugehörige Code nicht geändert werden kann, z.B. Funktionen des Betriebssystems
- Frameworks, z.B. Mockito, EasyMock, JMockit (Java) bzw. Moq, Nsubstitute, FakeItEasy, JustMock (C#)

# Komponententest: Empfehlungen

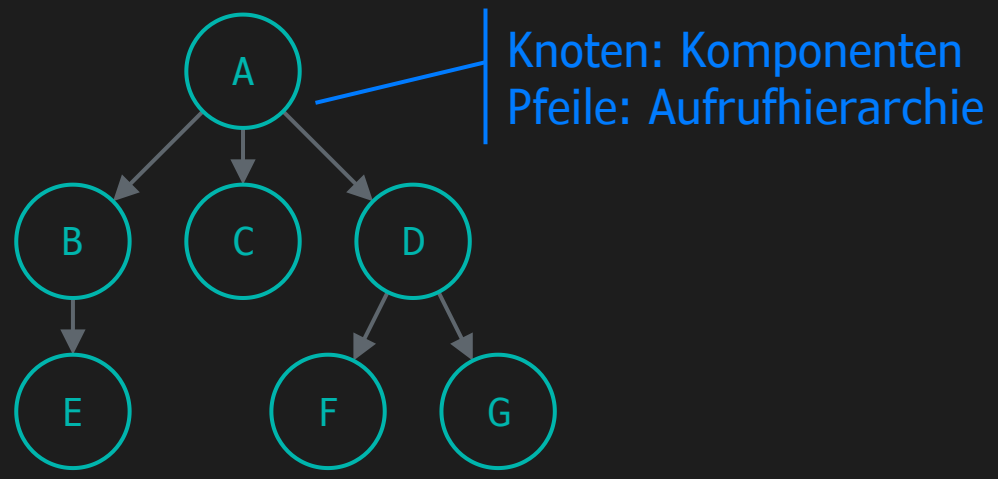
- Merkmale eines guten Komponententests
  - Schnell: normalerweise werden viele Komponententests ausgeführt
  - Isoliert: keine Abhängigkeiten von externen Faktoren (z.B. Datenbank)
  - Wiederholbar: liefern gleiche Ergebnisse, falls keine Code-Änderungen
  - Autark: Ausführung ohne menschliche Interaktion
  - Zeitig: das Schreiben des Tests sollte nicht zu lange dauern
  - Aussagekräftige Namen der Tests
  - Pro Test immer nur ein Aspekt: sonst Gefahr der Fehlermaskierung
  - Logik im Test vermeiden
  - Parametrisierung der Daten
  - Überprüfen privater Methoden durch Tests öffentlicher Methoden
- AAA-Pattern
  - Arrange: Vorbereitung des Testobjekts & der Parameter
  - Act: Ausführung des Tests
  - Assert: Überprüfung des Ergebnisses (bestanden / fehlgeschlagen)
- Codeabdeckung (Code Coverage) messen, z.B. »mindestens 80%«

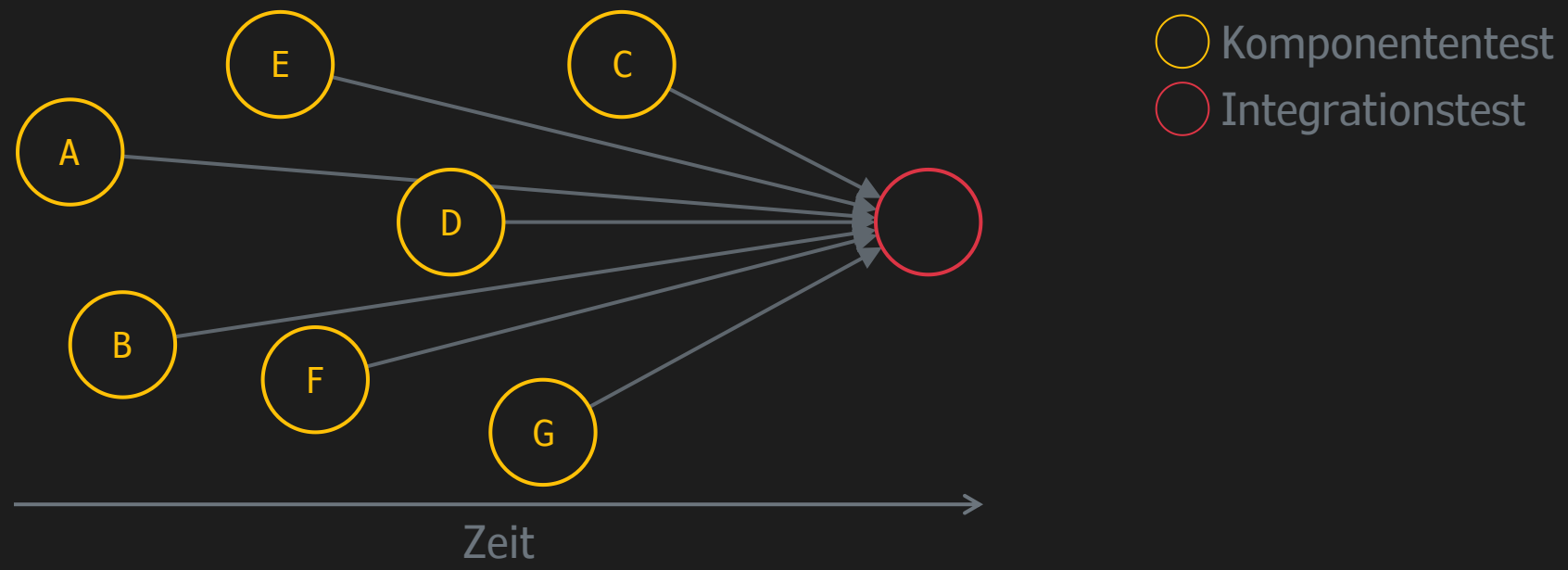
# Integrationstest

- Voraussetzung: Komponententests abgeschlossen
- Testziele
  - Fehlerzustände in Schnittstellen / im Zusammenspiel der integrierten Komponenten (Bausteine) aufdecken
  - Verifizierung des korrekten Verhaltens der Schnittstellen
- Testbasis
  - Komponentenübergreifende UML-Modelle, z.B. Anwendungsfälle, Komponenten-, Aktivitäts-, Zustands-, Sequenzdiagramme
  - Spezifikationen von Schnittstellen (API) & Kommunikationsprotokolle
- Testumgebung
  - Wie bei Komponententest, aber zusätzlich Monitoring für Überwachung von Schnittstellen
- Testfälle gemäß Integrationsstrategie (Reihenfolge der Integration), z.B. Big-Bang, Top-Down, Bottom-Up

# Integrationsstrategie: Big-Bang

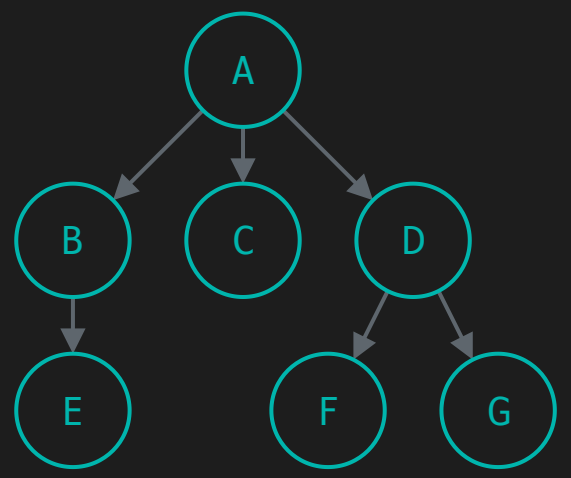
- Integration erst, wenn alle Komponenten fertig sind
- Tests sehr spät: Wartezeit bleibt ungenutzt
- Alle Fehlerwirkungen treten zusammen auf

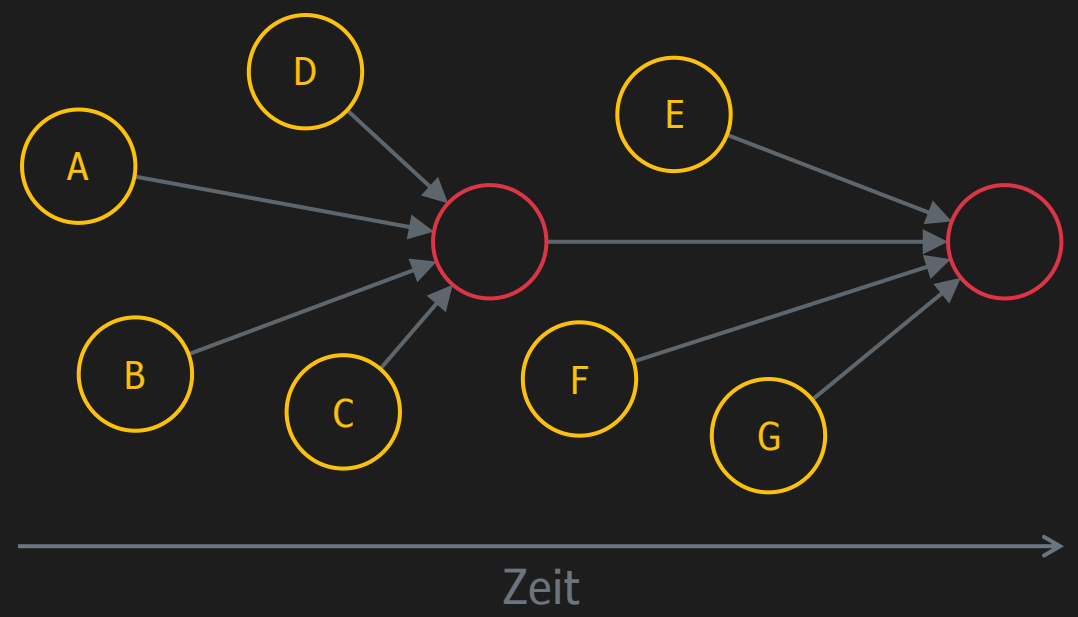


# Integrationsstrategie: Top-Down

- Untergeordnete Komponenten zunächst durch Platzhalter
- Platzhalter werden von oben nach unten durch bereitgestellte Komponenten ersetzt
- Übergeordnete Schichten dienen als Treiber



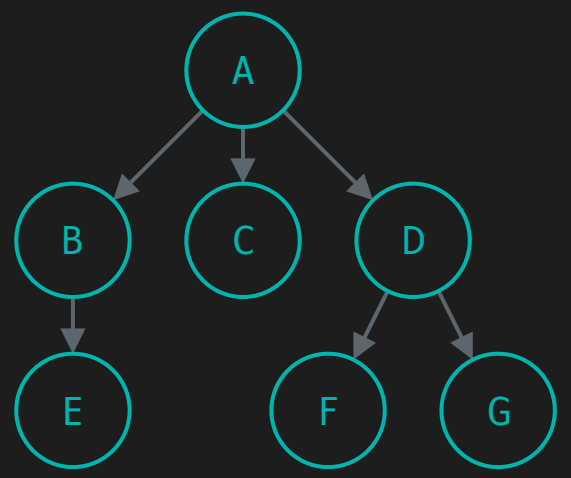



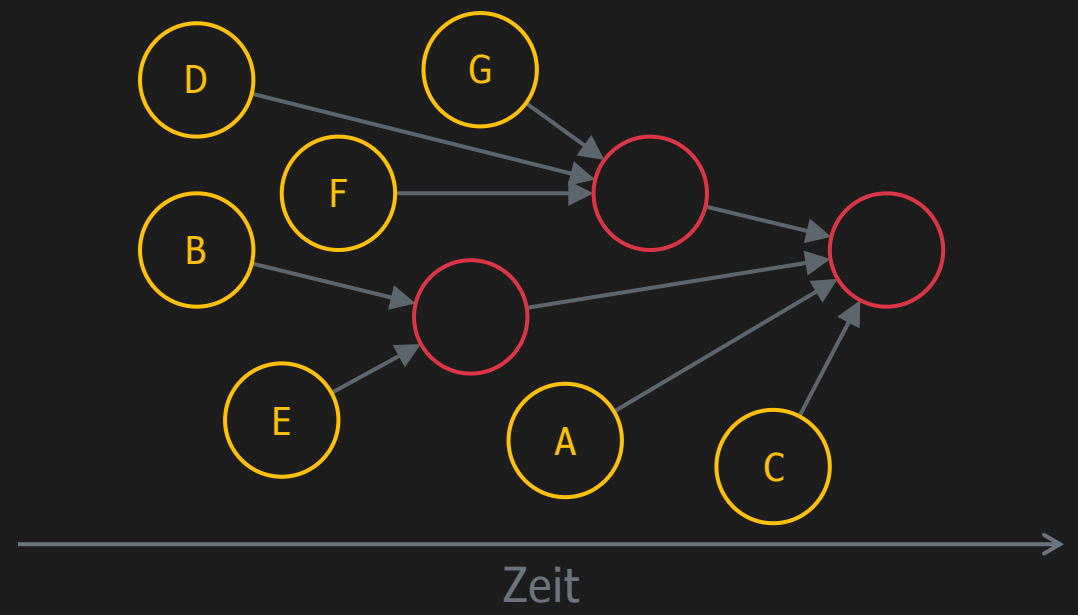

Komponententest

Integrationstest

# Integrationsstrategie: Bottom-Up

- Tests beginnen bei den Blättern
- Übergeordnete Komponenten werden durch Treiber simuliert
- Sukzessive Integration und Tests von unten nach oben




Komponententest

Integrationstest

# Systemtest

- Voraussetzung: Integrationstests abgeschlossen
  - Komponenten- und Integrationstests: Prüfung aus Sicht der Auftragnehmenden
  - Systemtest: Prüfung aus Sicht der Auftraggebenden bzw. Nutzenden
- Testziele
  - Fehlerzustände im Zusammenspiel aller Komponenten aufdecken
  - Verifizierung des korrekten Verhaltens des Systems
  - Validierung, ob System für Einsatzzweck tauglich
  - Vorbereitung der Abnahme bzw. Freigabe des Systems
- Testbasis
  - Funktionale & nicht-funktionale Anforderungen
  - Systemweite Modellierung, z.B. Anwendungsfälle, Komponenten-, Aktivitäts-, Zustands-, Sequenzdiagramme; Epics & User Stories
- Testumgebung
  - Nahe an Produktivumgebung; keine Treiber oder Platzhalter mehr

# Systemtest: Herausforderungen

- Aus Kostengründen: System- oder Abnahmetest oft in Produktivumgebung
  - Fehlerwirkungen können zu teuren Schäden oder Datenverlust führen
  - Testbedingungen ungeeignet: kaum Kontrolle über Konfiguration
  - Systemtests oft nicht reproduzierbar
- Versäumte Entscheidungen
  - Unklare, implizite oder unterschiedliche Anforderungen der Stakeholder
  - Klärungs- und Entscheidungsprozesse noch beim Systemtest
  - »Kopf in den Sand«-Syndrom: obwohl schon absehbar, wird das Scheitern des Projekts erst beim Systemtest offiziell attestiert

# Abnahmetest

- Voraussetzung: Systemtest abgeschlossen
  - Spezielle Form des Systemtests vor Inbetriebnahme
  - Wird im Allgemeinen von und bei Auftraggebenden durchgeführt
  - Vertraglicher Betriebstest bzw. Abnahmetest der Nutzenden (User Acceptance Test, UAT)
- Testziele
  - Vertragliche Erfüllung; rechtliche & regulatorische Anforderungen
  - Verifizierung des korrekten Verhaltens des Systems (fehlerfrei)
  - Validierung, ob System für Einsatzzweck tauglich ist (mangelfrei)
- Testbasis wie bei Systemtest plus
  - Geschäftsprozesse, Vorschriften, rechtliche Verträge & Standards
  - System- & Nutzungshandbuch, Installationsverfahren
  - Verfahren für Sicherung, Wiederherstellung, Disaster Recovery
- Testumgebung: spezielle Abnahmeumgebung bei Auftraggebenden

# Alpha- & Beta-Test

- Systemtests für alle möglichen Umgebungen: sehr kostenintensiv oder unmöglich
- Alpha-/Beta-Test vor dem Abnahmetest
  - Lieferung stabiler Vorabversion an repräsentative Nutzende
  - Durchführung vorgegebener Tests; Probe-Einsatz unter realistischen Bedingungen; Feedback durch Nutzende
  - Achtung: sollte Systemtest nicht ersetzen!
- Testziele
  - Fehler durch unbekannte Umgebungen identifizieren
  - Vertrauen bei Stakeholdern aufbauen, dass System einsatzfähig ist
- Alpha-Test
  - Auf Systemen der Auftragnehmenden, aber durch Nutzende oder unabhängiges Testteam
- Beta-Test (aka Feldtest)
  - Auf Systemen der Auftraggebenden

# Wartungstest

- Wartungstest, wenn System bereits im Betrieb
- Anlass: Modifikation
  - Additive Wartung: Geplante Weiterentwicklung & Releases
  - Adaptive Wartung: Änderung der Umgebung
  - Korrektive Wartung: Fehlerbehebung, Notfallkorrekturen, Hot Fix
  - Perfektive Wartung: Verbesserung der Qualität
- Anlass: Migration
  - Plattform, Daten, Hardware, DB-Systeme, Webserver, Frameworks
  - Authentisierungs- und Verzeichnisdienste, Netzwerkdienste
  - Dateiablage, Druckdienste, Überwachungs- und Management-Dienste
- Anlass: Außerbetriebnahme
  - Ablösung des Systems
  - Archivierung inkl. Datenmigration

# Abschnitt 6.3

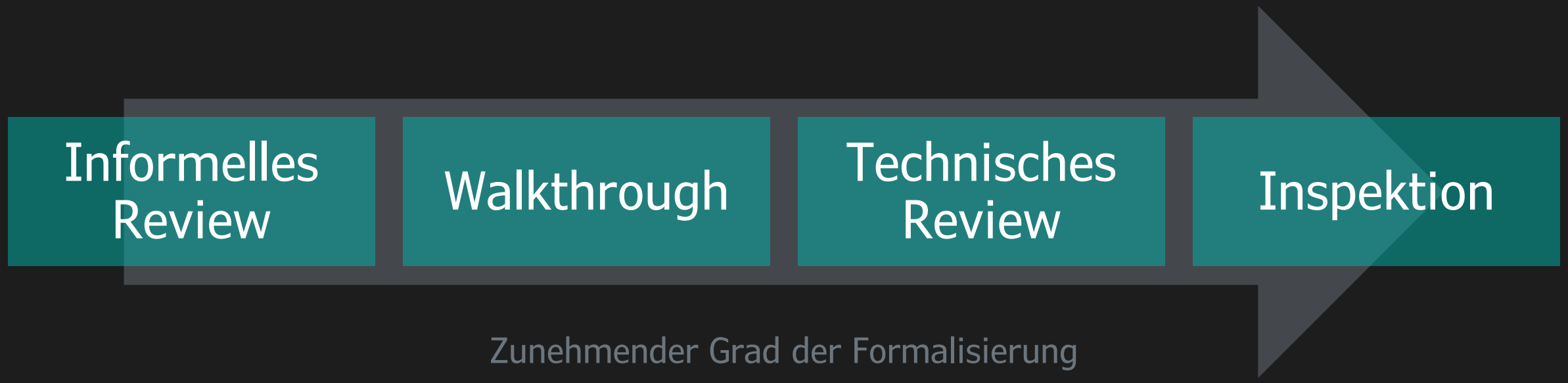
## Statische Tests

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

# Statische Tests

- System wird beim Test nicht ausgeführt
- Testobjekt: Document under Review (DUR), z.B.
  - Verträge, Projekt-, Zeit- und Budgetpläne
  - Spezifikation von Anforderungen, Epics, User Stories etc.
  - Architektur, Entwurfsspezifikationen, UML-Diagramme
  - Quellcode
  - Nutzungshandbücher
  - Testkonzepte, Testspezifikationen, Testskripte, Testberichte etc.
- Findet Fehlerzustände (Ursachen)
- Ziel: Prävention, Fehlerzustände so früh wie möglich beheben
- Statische Tests sollten vor dynamischen Tests erfolgen
- Formen
  - Reviews: Prüfungen durch eine / mehrere Personen
  - Code-Analysen, Kontroll- und Datenflussanalyse, Metriken

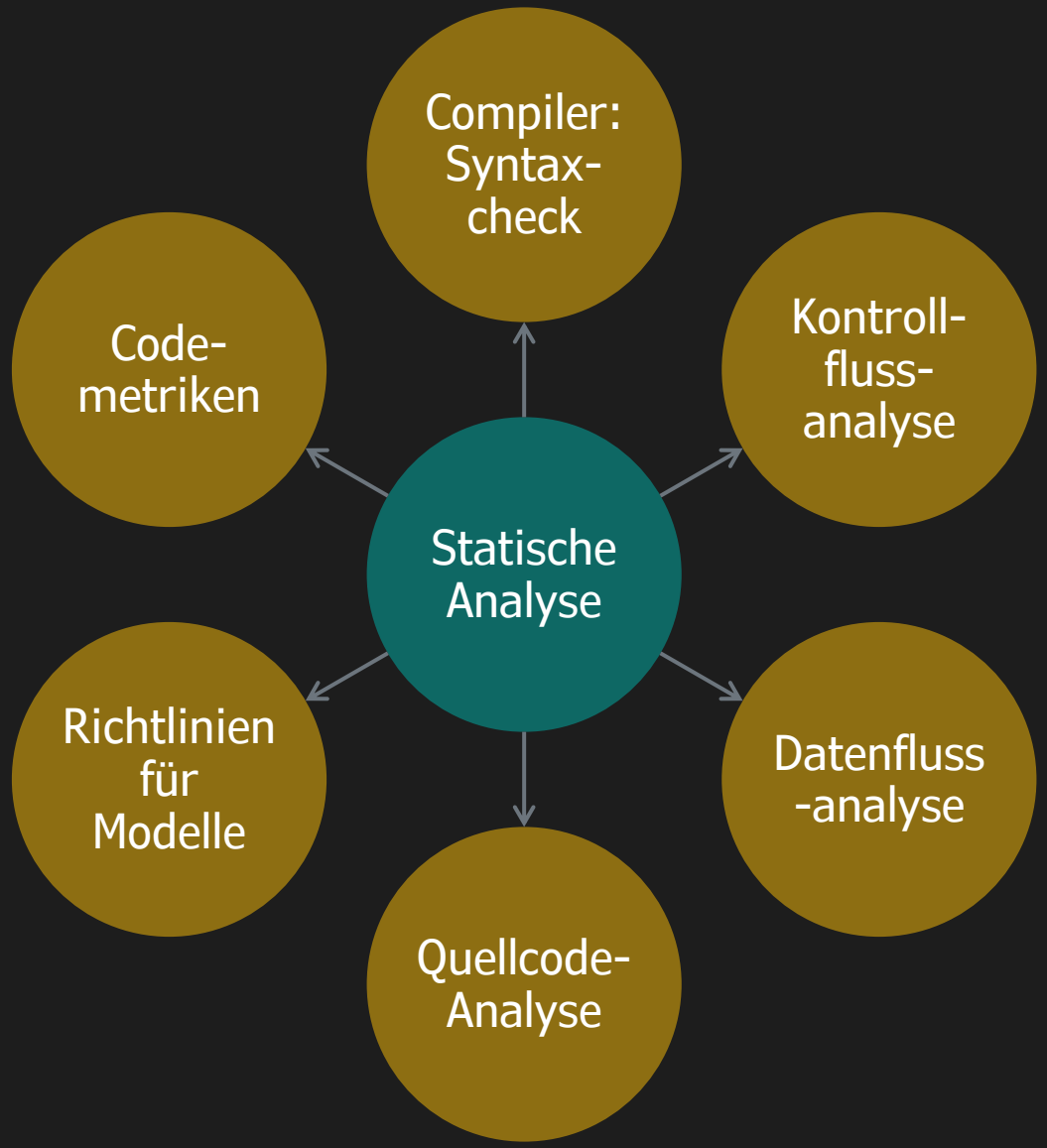
# Formen von Reviews




- Studien
  - Reviews machen ca. 10-15% des Entwicklungsbudgets aus, bewirken aber ca. 14-25% Einsparungen (Kosten schon abgezogen)
  - Bis zu 70% der Fehler werden dabei entdeckt

Quelle: (ISO/IEC 20246:2017; Spillner, Linz 2024)

# Statische Analyse: Werkzeugunterstützung

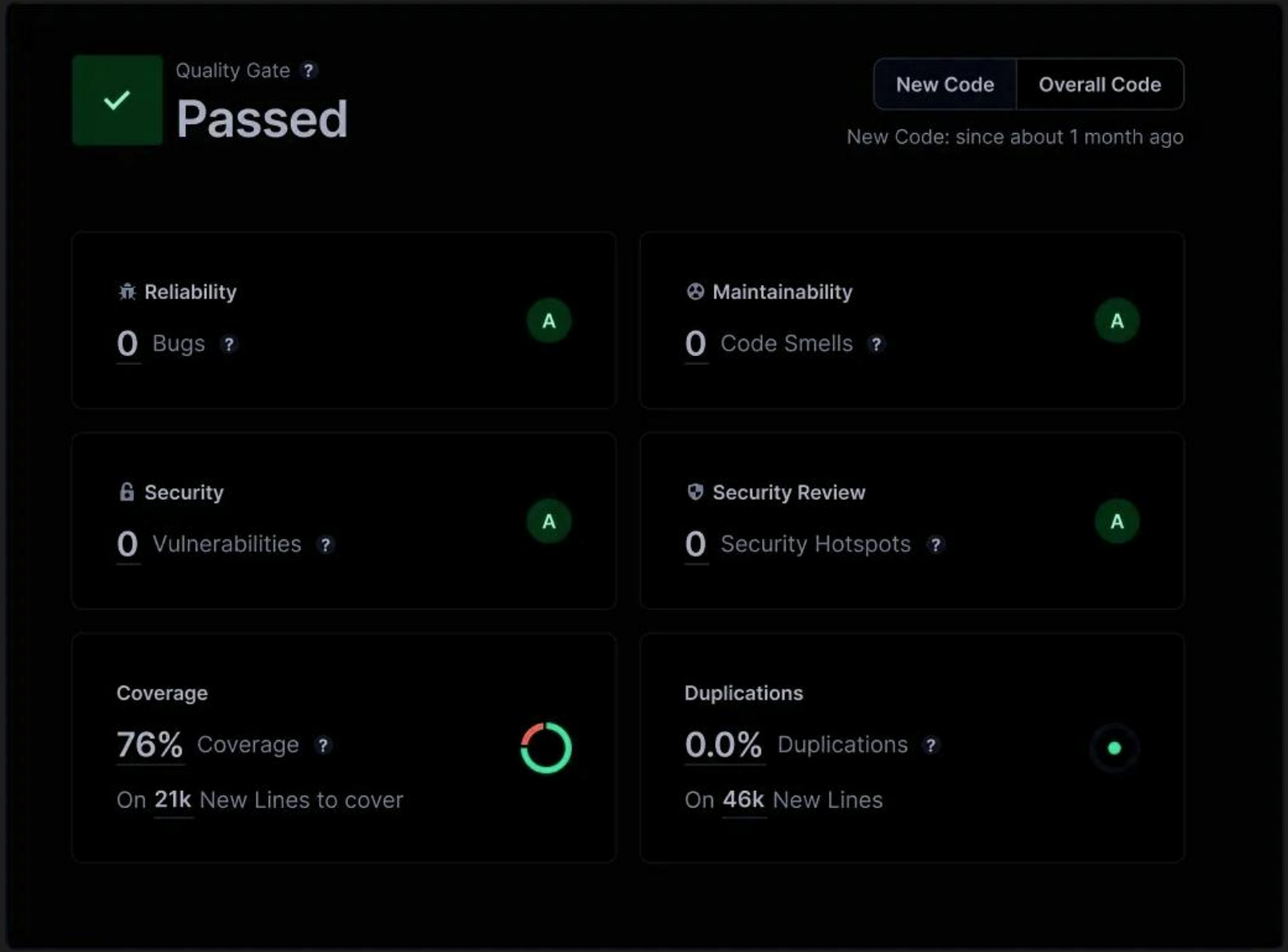


# Statischen Analyse: Beispiele

| Werkzeug | URL | Sprachen |
|---|---|---|
| SonarCube | https://www.sonarsource.com | Viele gängige Sprachen |
| Lattix | https://www.lattix.com | Java, C/C++, C#, JavaScript, Python u.a. |
| Sonargraph | https://www.hello2morrow.com | Java, C/C++, C# |
| Structure101 | https://structure101.com | Java, C#, Plugins für C/C++, PHP |
| Teamscale | https://teamscale.com | Java, C/C++, C#, ABAP, JavaScript, Python |
| ArchUnit | https://www.archunit.org | Unit Tests für Architekturen, z.B. Java, C# |

- Statische Analyse des Quellcodes: Abhängigkeiten, Zyklen, Schichten, Bausteine (z.B. Pakete bzw. Namespaces), Schnittstellen
- Prüfung von Anti Patterns, Code Smells, Sicherheitsaspekten etc.
- Berechnung von spezifischen Softwaremetriken
- Teilweise Prüfung gegen formale Architekturbeschreibung, z.B. Schichten
- Einbindung in CI-Pipeline als »Continuous Inspection«

# Werkzeug: SonarQube



# Kontrollflussanalyse

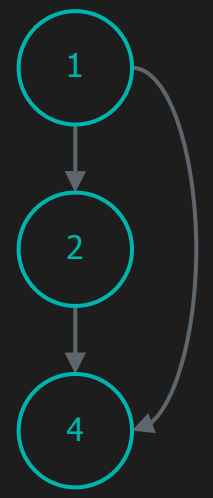
- Kontrollfluss einer Operation (Methode, Prozedur etc.)
  - Abstrakte Repräsentation von Quellcode als Abfolge von Anweisungen
  - Wird durch »steuernde« Anweisungen determiniert
    - Sprünge: goto
    - Bedingte Verzweigungen: if-then-else, switch-case
    - Schleifen: while, do-while, for
- Kontrollflussgraph (Control-Flow Graph, CFG)
  - Gerichteter Graph, verwandt mit Programmablaufplan (PAP)
    - Knoten: Anweisungen im Quellcode
    - Kanten: Folgeanweisung; mehrere bei Verzweigungen
    - Genau ein Start- und ein Endknoten
  - Sprünge aus Schleifen heraus / in Schleifen hinein sind möglich
- Kontrollflussgraph stellt alle »wesentlichen Abläufe« einer Operation in abstrakter Form dar; ermöglicht graphentheoretische (und visuelle) Prüfung von Code

# Kontrollflussgraphen

- Für Operationen
  - Gemäß strukturierter Programmierung als Grundform der imperativen Programmierung
  - Im Kern: wie Übersetzung des Compilers in Assembler / Maschinencode
  - Kontrollstrukturen werden in Sprünge übersetzt
  - Konstrukte aus höheren Programmiersprachen müssen entsprechend transformiert werden
- Regeln
  - Einstiegspunkt der Operation immer als ein separater Startknoten
  - Keine Modellierung einer Parameterübergabe
  - Ausstiegspunkt der Operation immer als ein separater Endknoten; auch, falls Operation ohne Rückgabe (void)
  - Einzelne Anweisungen als Knoten; ein Knoten pro Anweisung
  - Ketten von Anweisungen ohne Verzweigung / Schleifen (»Anweisungsblock«) können zu einem Knoten zusammengefasst werden
  - Beachte: auch Knoten für Bedingungsprüfung einer Verzweigung oder Schleife

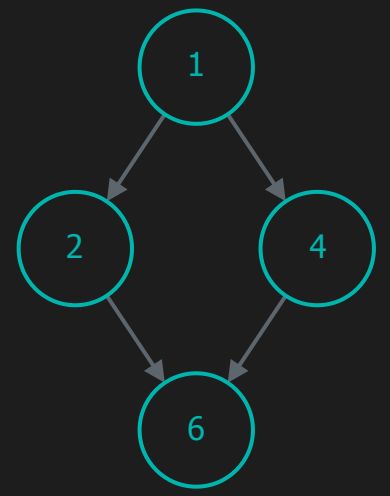
# Kontrollflussgraphen: Verzweigungen

```
1: if (_) {
2:   _;
3: }
4: _;
```



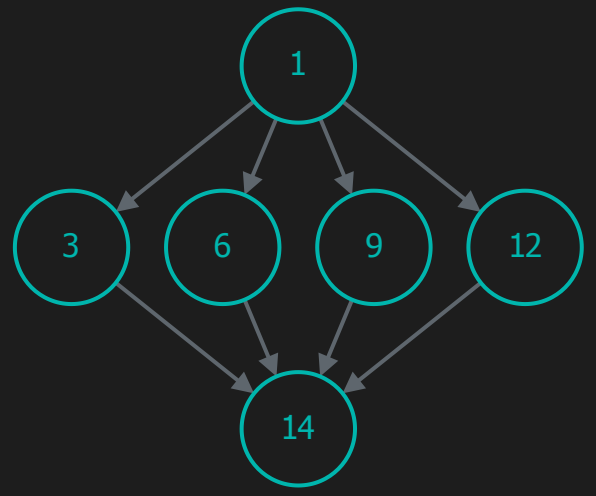

If-Anweisung

```
1: if (_) {
2:   _;
3: } else {
4:   _;
5: }
6: _;
```




If-Else-Anweisung

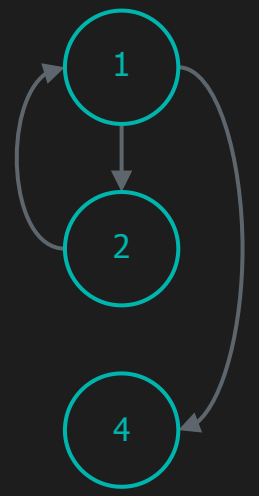
```
1: switch (_) {
2:   case 1:
3:     _;
4:     break;
5:   case 2:
6:     _;
7:     break;
8:   case 3:
9:     _;
10:    break;
11:  default:
12:    _;
13: }
14: _;
```




Switch-Anweisung

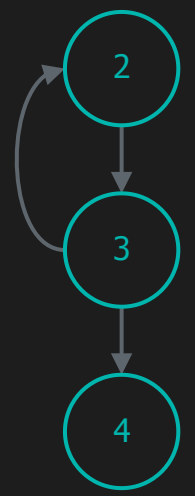
# Kontrollflussgraphen: Schleifen

```
1: while (_) {
2:   _;
3: }
4: _;
```




While-Schleife

```
1: do {
2:   _;
3: } while (_);
4: _;
```

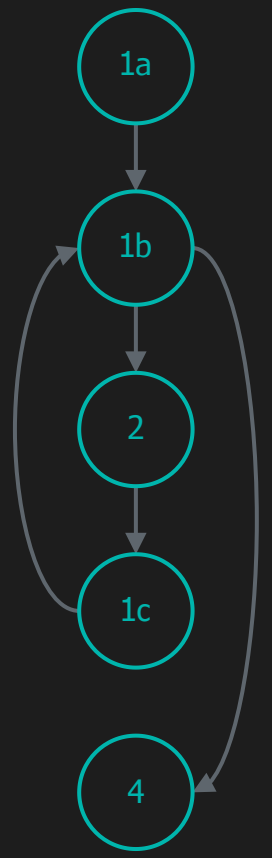



Do-While-Schleife

```
1: for (init; condition; iterator) {
2:   _;
3: }
4: _;
```

```
1a: init;
1b: while (condition) {
 2:   _;
1c:   iterator;
 3: }
 4: _;
```

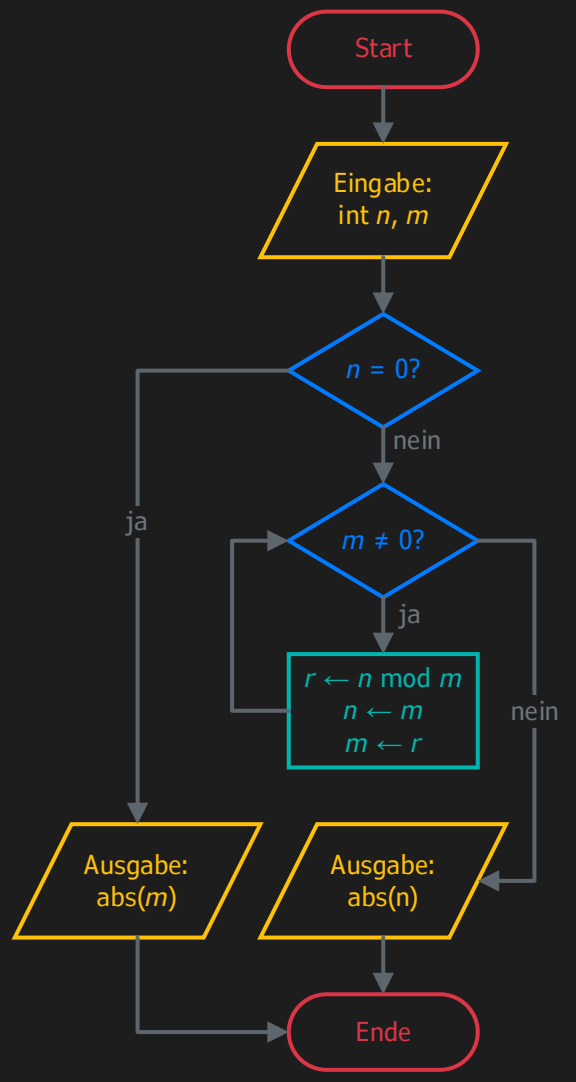
For-Schleife mit Compiler-Übersetzung



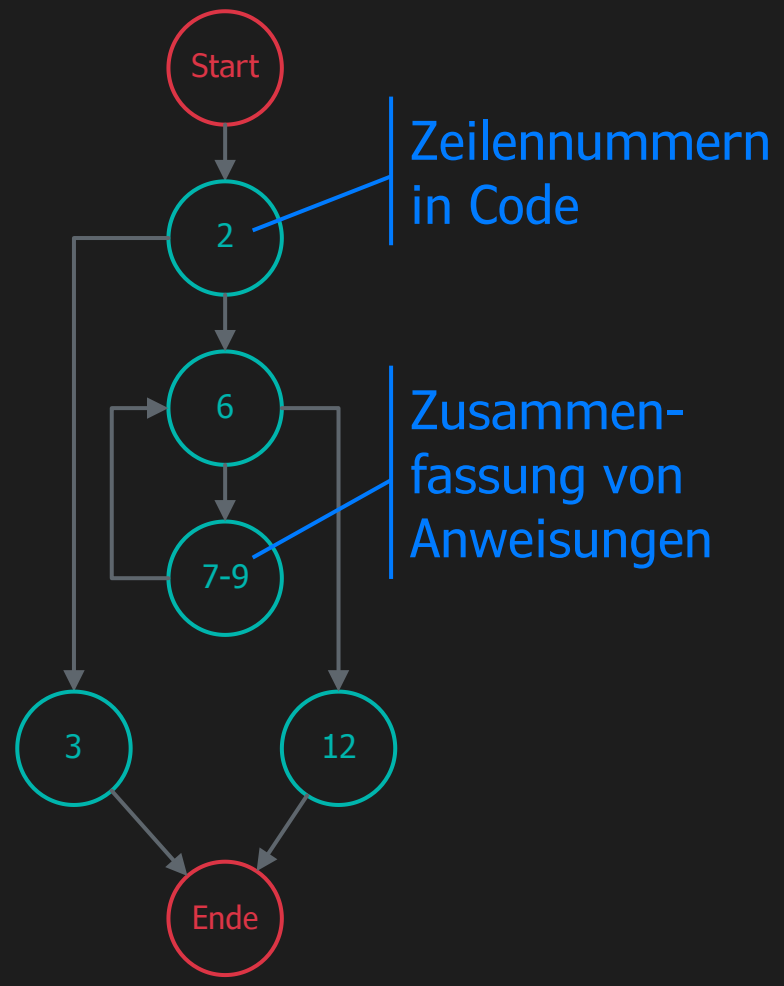
# Euklidischer Algorithmus: Code

```
1: int greatestCommonDivisor(int n, int m) {
2:     if (n == 0) {
3:         return Math.abs(m);
4:     }
5:
6:     while (m != 0) {
7:         final int r = n % m;
8:         n = m;
9:         m = r;
10:    }
11:
12:    return Math.abs(n);
13: }
```

# Euklidischer Algorithmus: Kontrollflussgraph




Programmablaufplan (PAP)




Kontrollflussgraph (CFG)

# Softwaremetriken

- Quantifizierbare Prozess- oder Produkteigenschaften bei der Softwareentwicklung
  - Helfen bei Analyse, Bewertung, Planung & Überwachung
  - Siehe auch Key Performace Indicator (KPI)
- Prozessmetrik, z.B.
  - Projekt: Dauer, Aufwand, Kosten
  - Requirements Engineering: Anzahl Anforderungen, Änderungsanträge
  - Scrum: Sprintlänge, Velocity
  - DevOps: Release-Abstände, Mean Time To Change (MTTC), Mean Time to Recovery (MTTR), Mean Time Between Failures (MTBF)
- Diverse Codemetriken, siehe nächste Folie

Quelle: (Witte 2018)

# Codemetriken: Beispiele

| Metrik | Ebene |
|---|---|
| Lines of Code (LOC) | Projekt |
| Code Duplication | Projekt |
| Test Coverage | Projekt |
| Number of Children (NOC) | Komponente |
| Depth of Inheritance Tree (DIT) | Komponente |
| Coupling between Objects (CBO) | Komponente |
| Cumulative Component Dependency (CCD) | Komponente |
| Lack of Cohesion of Methods (LCOM) | Klasse |
| Zyklomatische Komplexität (aka McCabe-Zahl) | Methode |
| Halstead-Metrik | Methode |
| Maintainability Index (MI) | Methode |

Quelle: (Ciceri, Farley, Ford, Harmel-Law, Keeling, Lilienthal, Rosa, Zitzewitz, Weiss, Woods 2022)

# Zyklomatische Komplexität nach McCabe (1976)

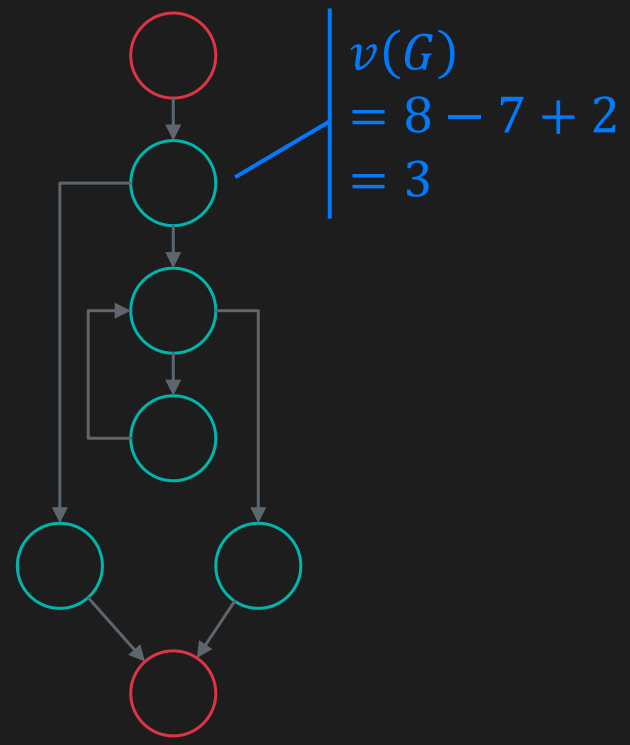
- Graphentheorie
  - Anzahl Kanten eines stark-zusammenhängenden Graphen, die entfernt werden müssen, um einen »Spannbaum« zu erhalten
- Für Kontrollflussgraphen $G$ (ohne unerreichbaren Code)
  - Anzahl »linear unabhängiger« Pfade von Start- zu Endknoten, aus denen durch Kombination alle Pfade erzeugt werden können
  - Jede Verzweigung & jede Schleife erhöht diese Komplexität
  - Hohe Komplexität für Menschen nur schwer nachvollziehbar
  - Entspricht Mindestzahl an Testfällen für vollständige Zweigüberdeckung (siehe unten)
- Für $G$ mit $e$ Kanten und $n$ Knoten gilt

$$v(G) = e - n + 2$$

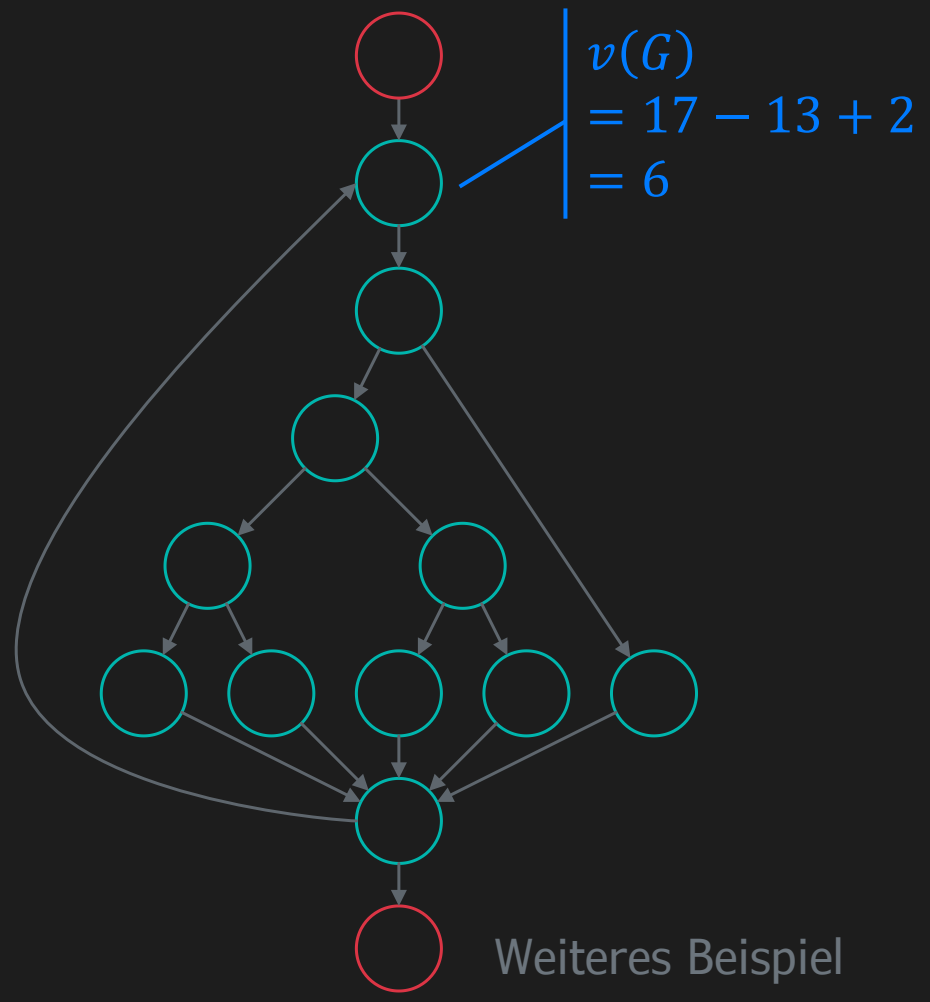
- $v(G) = 1$ entspricht linearem Programmablauf ohne Verzweigungen und Schleifen
- Beachte: der Wert ändert sich nicht durch das Zusammenfassen von Anweisungsblöcken

Quelle: (McCabe 1976)

# Zyklomatische Komplexität: Beispiele




Euklidischer Algorithmus




Weiteres Beispiel

- Kritikpunkte
  - Switch-Anweisung hat hohe Komplexität, ist aber intuitiv
  - Komplexität der Anweisungen bleibt unberücksichtigt

# Softwaremetriken: Empfehlungen

| Metrik | Wertebereich |
|---|---|
| Anzahl Felder (Instanzvariablen) pro Klasse | Max. 10 |
| Methoden pro Klasse | 3 – 15 |
| Parameter pro Methode | Max. 6 |
| LOC pro Methode | Max. 20 |
| Zyklomatische Komplexität pro Methode | Max. 15 |
| Abhängigkeit der Klasse von anderen Klassen | Max. 4 |
| MI | 20 – 100 |
| DIT | Max. 6 |
| CBO | Max. 9 |
| Test Coverage | 70% – 90% |
| Code Duplication | Max. 10% |

- Achtung: unterschiedliche Auffassungen in der Literatur; immer kontextabhängig; begründete Ausnahmen sind möglich

# Abschnitt 6.4

## Dynamische Tests

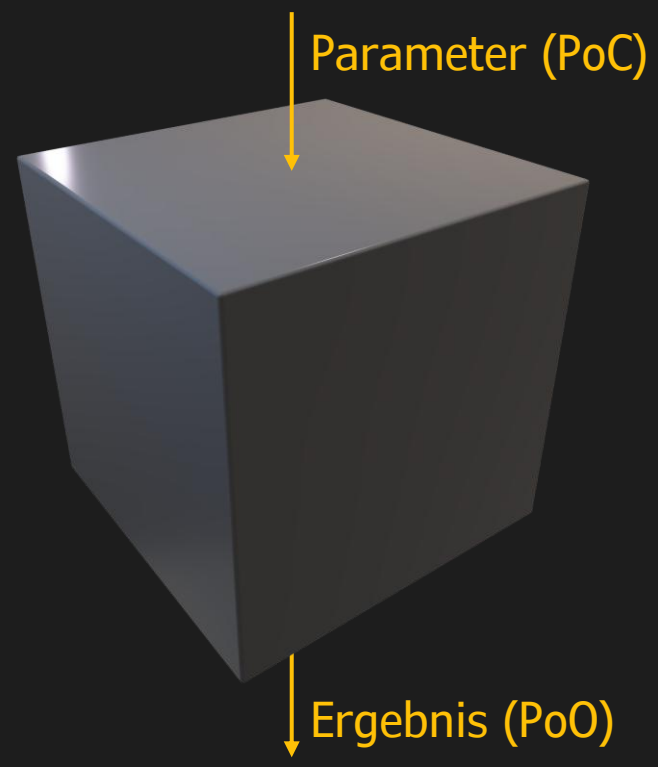
FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

# Dynamische Tests

- System wird beim Test ausgeführt
- Prüfung des Verhaltens, insbesondere Rückgabe von Operationen für spezifische Parameter (ggf. mit Vor- & Nachbedingungen)
- Testobjekt: System under Test (SUT)
- Findet Fehlerwirkungen (Symptome)
- Ziel
  - Nachweis: Erfüllung der Anforderungen durch stichprobenhafte Systemläufe in einer speziellen Testumgebung, ggf. unter Einsatz von Platzhaltern und Treibern
  - Parameter / Steuerung über Point of Control (PoC)
  - Beobachtung der Ergebnisse / des Verhaltens über Point of Observation (PoO)
- Dynamische Tests sollten nach statischen Tests erfolgen
- Herausforderung
  - Vollständiges Austesten aller Kombinationen in der Regel nicht möglich, nur Stichproben
  - Welche Parameter sind aussagekräftig?

# Dynamische Tests: Formen

## Black-Box-Test




Innere Struktur des Codes ist nicht bekannt

## White-Box-Test

Parameter

```
int greatestCommonDivisor(int n, int m) {
    if (n == 0) {
        return Math.abs(m);
    }

    while (m != 0) {
        final int r = n % m;
        n = m;
        m = r;
    }

    return Math.abs(n);
}
```

PoC & PoO

Ergebnis

Innere Struktur des Codes ist bekannt

# ISO/IEC/IEEE 29119: Testtechniken

## Black-Box-Test

- Äquivalenzklassenbildung (Equivalence Partitioning)
- Klassifikationsbaumverfahren (Classification Tree Method)
- Grenzwertanalyse (Boundary Value Analysis)
- Syntaxtest (Syntax Testing)
- Kombinatorisches Testen (Combinatorial Test Design Techniques)
- Entscheidungstabellentest (Decision Table Testing)
- Ursache-Wirkungs-Graph-Analyse (Cause Effect Graphing)
- Zustandsübergangstest (State Transition Testing)
- Anwendungsfallbasierter Test (Scenario Testing)
- Zufallstest (Random Testing)
- Metamorphes Testen (Metamorphic Testing)
- Anforderungsbasiertes Testen (Requirement-based Testing)

## White-Box-Test

- Anweisungstest (Statement Testing)
- Zweigtest (Branch Testing)
- Entscheidungstest (Decision Testing)
- Bedingungstest (Branch Condition Testing)
- Mehrfachbedingungstest (Branch Condition Combination Testing)
- Modifizierte Bedingungs-/Entscheidungsüber-deckung (Modified Condition/Decision Coverage Testing, MC/DC)
- Datenflusstest (Data Flow Testing)
- Intuitive Testfallermittlung (Error Guessing)

Quelle: (ISO/IEC/IEEE 29119-4:2021)

# Black-Box: Äquivalenzklassenbildung

- Partitionierung der Operationsparameter in Bereiche (Äquivalenzklasse, ÄK)
  - ÄK sind paarweise disjunkt; Vereinigung aller ÄK ergibt gesamten Definitionsbereich
  - Beliebiges Element einer ÄK zeigt »gleiches Testverhalten« wir alle anderen Elemente dieser ÄK, d.h. jedes Element kann Repräsentant der ÄK für einen Testfall sein
- Beachte
  - Äquivalenzklassenbildung ist nicht eindeutig
  - Möglich von »sehr fein« (jeder Parameterwert bildet eigene ÄK) bis »sehr grob« (eine ÄK für gesamten Definitionsbereich)
  - Ziel: möglichst wenige, aber aussagekräftige ÄK, die jeweils bestimmtes Verhalten abbilden
  - Eine ÄK sollte entweder nur gültige Parameterwerte (Operation arbeitet normal / liefert Rückgabe) oder nur ungültige Parameterwerte (Operation wirft Fehler) enthalten; diese werden entsprechend als gültige ÄK bzw. und ungültige ÄK bezeichnet
  - Bei numerischen Datentypen üblicherweise Intervalle
- Testverfahren: partitioniere Definitionsbereich in ÄK & wähle für jede ÄK einen Repräsentanten als Testfall
- Metrik: Anteil getesteter Äquivalenzklassen von allen Äquivalenzklassen in %

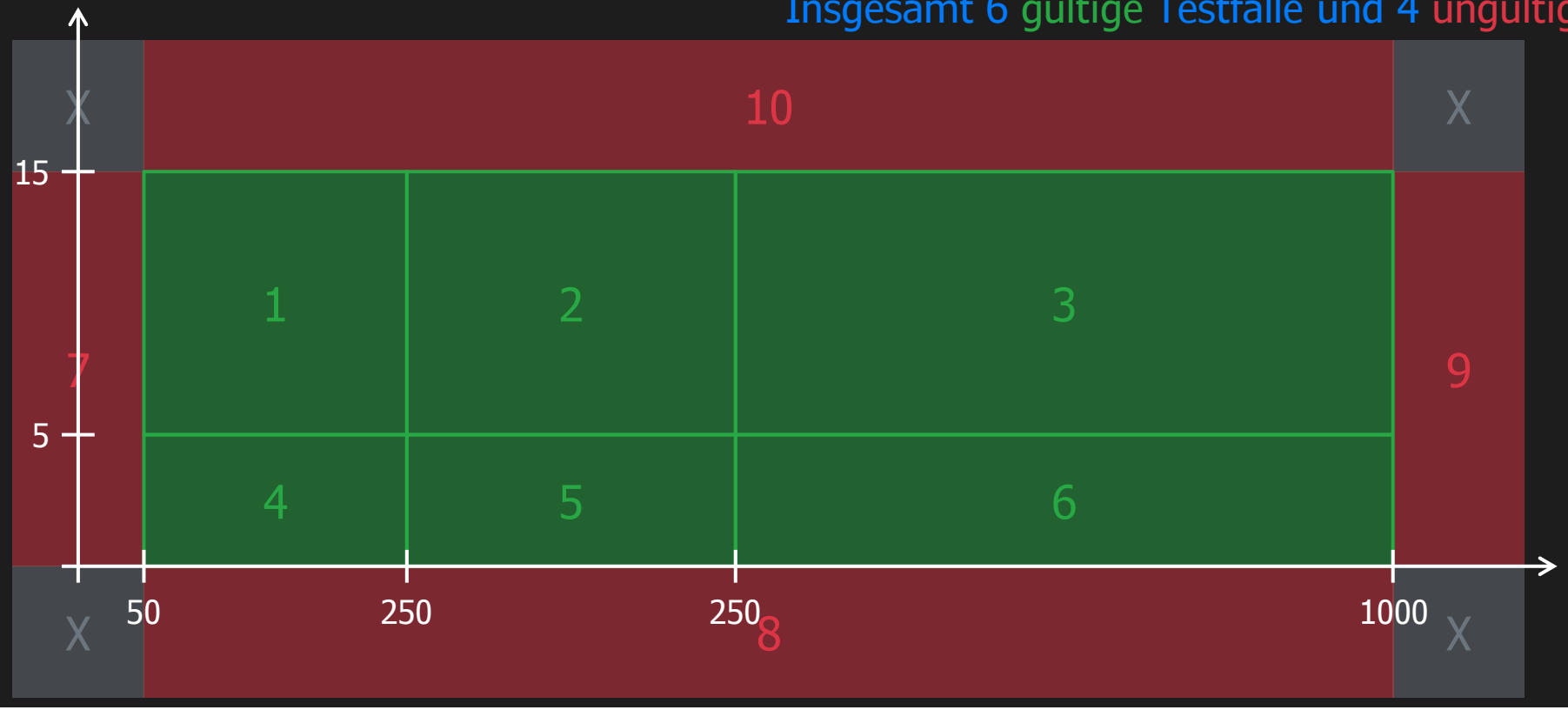
# Black-Box: Äquivalenzklassenbildung für mehrere Parameter

- Falls Parameter im Verhalten »unabhängig« voneinander sind
  - Bilde gültige und ungültige »eindimensionale« ÄK für jeden Parameter separat
  - Für alle Kombinationen von gültigen ÄK der einzelnen Parameter: erzeuge jeweils einen Testfall
  - Für jede ungültige ÄK eines Parameters: erzeuge einen Testfall durch Kombination mit jeweils einer beliebigen, gültigen ÄK der anderen Parameter, um Fehlermaskierung zu vermeiden
  - Anders formuliert: maximal eine ungültige ÄK bei Kombinationen der Parameter
  - Achtung: kombinatorische Explosion der gültigen ÄK; ggf. ist eine Beschränkung auf relevante Kombinationen oder eine Vergröberung der Partitionierung sinnvoll
- Falls Parameter im Verhalten »abhängig« sind
  - Bilde »mehrdimensionale« ÄK, die jeweils ein Verhalten abbilden
- Hinweis
  - Hier Annahme, dass statische Typisierung, d.h. ungültige Parameterwerte wie z.B. Text bei numerischen Datentypen sind schon durch den Compiler nicht möglich
  - Bei dynamisch typisierten Sprachen sind im Allgemeinen weitere ungültige ÄK nötig

# Äquivalenzklassenbildung: Beispiel für unabhängige Parameter

- Operation liefert Rabatt für eine Bestellung
  - Parameter »Bestellwert«: mindestens 50 €, höchstens 1.000 € (sonst Fehler); ab 250 € erste, ab 500 € zweite Rabattstufe
  - Parameter »$n$ Jahre Kunde«: Rabatt, wenn mindestens 5; Fehler, falls negativ oder größer 15
  - Die Rabatte sind unabhängig voneinander

Insgesamt 6 gültige Testfälle und 4 ungültige Testfälle



# Äquivalenzklassenbildung: Beispiel für unabhängige Parameter

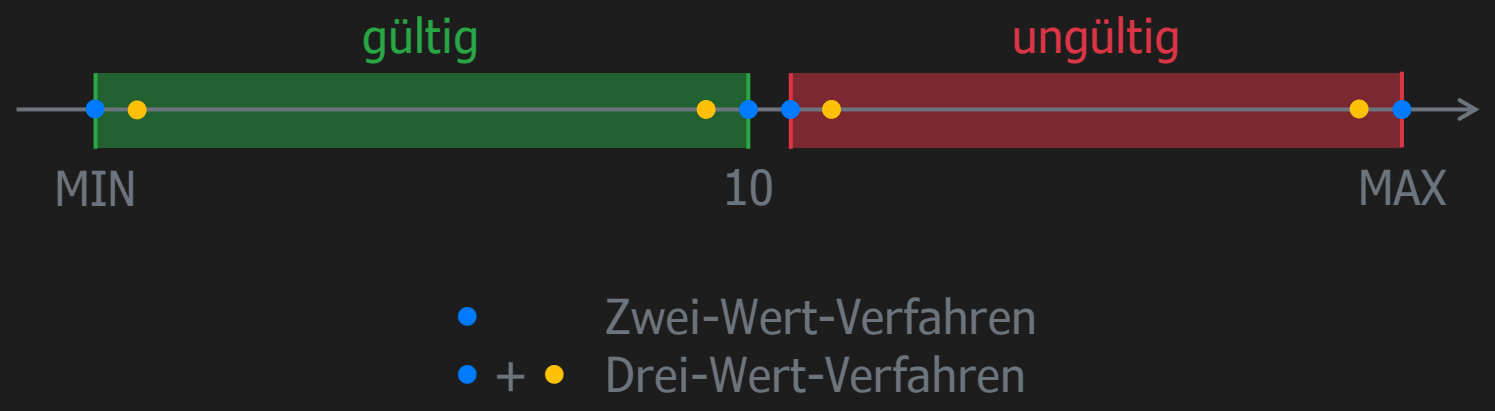
- Operation mit drei Parametern für Seitenlängen: diese prüft, ob die Parameterwerte ein rechtwinkeliges Dreieck bilden
- Beachte: die Parameter können nicht unabhängig voneinander betrachtet werden
- Mögliche Äquivalenzklassen
  - Ungültige ÄK: mindestens ein Parameter ist negativ (Fehler wird geworfen)
  - Gültige ÄK 1: Parameter nicht-negativ, aber keine Dreiecke, weil Dreiecksungleichung nicht erfüllt ist
  - Gültige ÄK 2: Dreiecke, aber nicht rechtwinkelig
  - Gültige ÄK 3: Rechtwinkelige Dreiecke

# Black-Box: Grenzwertanalyse (1)

- Beobachtung: Fehler treten vermehrt an Grenzen von Äquivalenzklassen auf (Off by One Error, OBOE)
- Beispiel Kundenstatus: 1.999 € → Silber, 2.000 € → Gold
- Grenzwertanalyse
  - Bestimme Testfälle in der Umgebung der Grenzen, bei denen Parameter gerade noch / gerade nicht mehr zu einer Äquivalenzklasse gehören
  - Betrifft nur geordnete Äquivalenzklassen, hier Fokus auf numerische Datentypen
  - Grenzwert: kleinster oder größte Wert einer geordneten Äquivalenzklasse
- Zwei-Wert-Verfahren
  - Für jeden Grenzwert einer Äquivalenzklasse wird dieser Grenzwert und der »nächste Nachbar« in der angrenzenden Äquivalenzklasse getestet
- Drei-Wert-Verfahren
  - Für jeden Grenzwert einer Äquivalenzklasse wird dieser Grenzwert und die beiden »nächsten Nachbarn« getestet

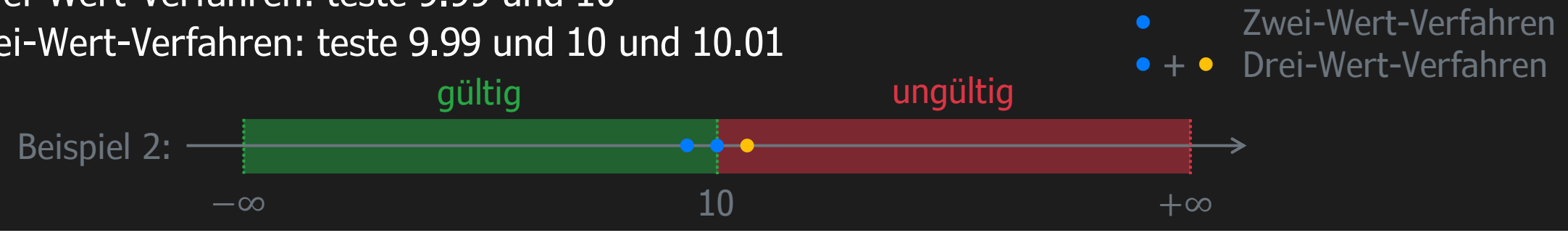
# Black-Box: Grenzwertanalyse (2)

- Integerzahlen: byte, short, int, long
  - Zahlenbereichs-Grenzen sind ebenfalls Grenzwerte, also auch MIN_VALUE bzw. MAX_VALUE
  - »Nachbar« durch Inkrement/Dekrement
- Beispiel: `if (x <= 10)` für Integer `x`
  - Zwei ÄK: [MIN_VALUE, 10] und [11, MAX_VALUE]
  - Zwei-Wert-Verfahren: teste MIN_VALUE, 10, 11, MAX_VALUE
  - Drei-Wert-Verfahren: teste MIN_VALUE, MIN_VALUE + 1, 9, 10, 11, 12, MAX_VALUE – 1, MAX_VALUE
  - Beachte: MIN_VALUE – 1 = MAX_VALUE bzw. MAX_VALUE + 1 = MIN_VALUE



# Black-Box: Grenzwertanalyse (3)

- Gleitkommazahlen: float, double
  - Zahlenbereichs-Grenzen sind keine Grenzwerte, also ohne NEGATIVE_INFINITY bzw. POSITIVE_INFINITY
  - Offenes Intervall: kein Grenzwert an offener Grenze
  - »Nachbar« durch Addition/Subtraktion eines geeigneten $\delta$ (hängt vom Kontext ab, hier 0.01)
- Beispiel 1: `if (x <= 10)` für Gleitkommazahl `x`
  - Zwei ÄK: ]NEGATIVE_INFINITY, 10] und ]10, POSITIVE_INFINITY[
  - Zwei-Wert-Verfahren: teste 10 und 10.01
  - Drei-Wert-Verfahren: teste 9.99 und 10 und 10.01
- Beispiel 2: `if (x < 10)` für Gleitkommazahl `x`
  - Zwei ÄK: ]NEGATIVE_INFINITY, 10[ und [10, POSITIVE_INFINITY[
  - Zwei-Wert-Verfahren: teste 9.99 und 10
  - Drei-Wert-Verfahren: teste 9.99 und 10 und 10.01

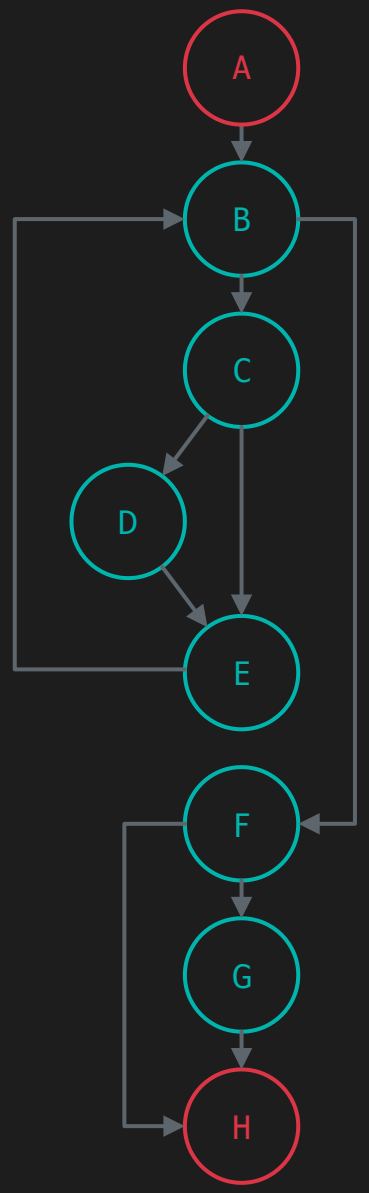


# Black-Box: Zustandsübergangstest

- Ziel: benutze UML-Zustandsdiagramm für Testobjekt, um Serien von Zustandsübergängen als Testfälle zu erzeugen
- Konformanztest
  - Testobjekt verhält sich bei erlaubten Transitionen konform zum Zustandsdiagramm, d.h. der erreichte Zustand entspricht dem Diagramm
- Robustheitstest
  - Testobjekt reagiert entsprechend bei unerlaubten Transitionen
- Technisch
  - Erzeuge »Transitionsbaum« ausgehend vom Startzustand, aber ohne Wiederholungen
  - Leite daraus Pfade von Wurzel zu Blättern für Konformanz- und Robustheitstests ab
  - Erzeuge Testfälle, die diese Pfade simulieren, wobei die auslösenden Ereignisse ausgelöst & die Wächterbedingungen erfüllt werden müssen

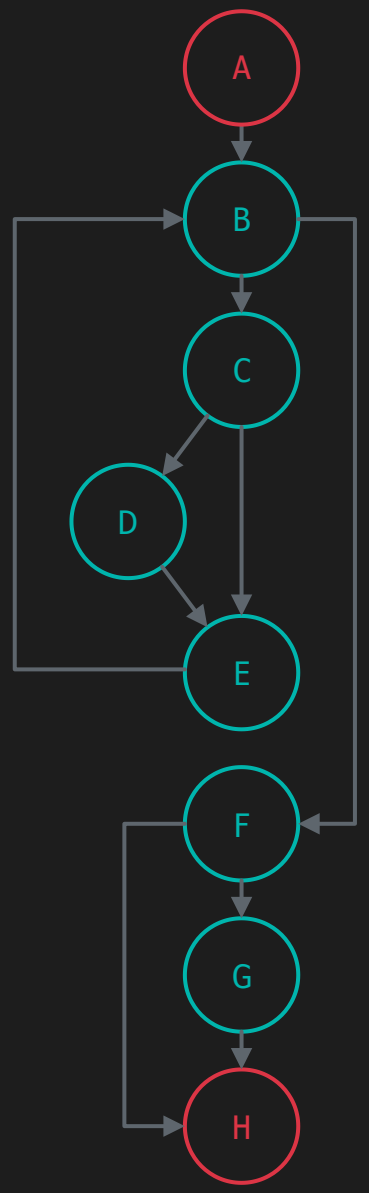
# White-Box-Test

- Herausforderung für Operationen
  - Finde Parameter, um möglichst alle relevanten Programmabläufe zu testen (als Pfade vom Start- zum Endknoten im Kontrollflussgraph)
- Verschiedene Ansätze und zugehörige Überdeckungsmetriken, z.B.
  - Anweisungstest ($C_0$)
  - Zweigtest ($C_1$)
  - Bedingungstest (oft mit $C_2$ bezeichnet)
  - Pfadtest (oft mit $C_\infty$ bezeichnet)
  - Grenze-Inneres-Test (oft mit $C_{BI}$ bezeichnet)
- Beachte
  - Die $C$-Notation ist in der Literatur nicht einheitlich
  - Es ist nicht immer möglich, alle theoretischen Pfade durch geeignete Parameter zu erzeugen
  - Für OOP nur bedingt geeignet, da Komplexität weniger durch einzelne Operationen, sondern eher durch Zusammenwirken von Klassen



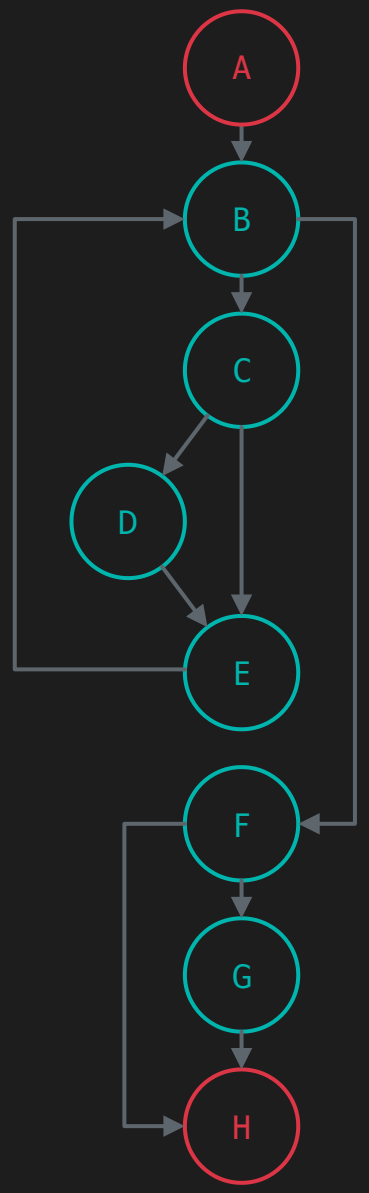
# White-Box: Anweisungstest

- Anweisungstest (Statement Testing)
  - Bestimme Testfälle, so dass jede Anweisung mindestens einmal ausgeführt wird
- Anweisungsüberdeckung (Statement Coverage) $C_0$
  - Anteil ausgeführter Anweisungen von allen (erreichbaren) Anweisungen in %
- Beispiel
  - Für Pfad A, B, F, G, H wäre Anweisungsüberdeckung $C_0 = \frac{5}{8} = 62{,}5\%$
  - Für $C_0 = 100\%$ genügt ein Testfall mit Pfad A, B, C, D, E, B, F, G, H
- Nachteile
  - Kontrollfluss und Abhängigkeit zwischen Daten werden nicht berücksichtigt
- Hinweis
  - Das Zusammenfassen von Anweisungsblöcken im Kontrollflussgraphen ändert eventuell den genauen Wert der Anweisungsüberdeckung, wenn diese weniger als 100% beträgt



# White-Box: Zweigtest

- Zweigtest (Branch Testing)
  - Bestimme Testfälle, so dass jeder Zweig (d.h. Kante) mindestens einmal durchlaufen wird
- Zweigüberdeckung (Branch Coverage) $C_1$
  - Anteil durchlaufener Zweige von allen Zweigen in %
- Beispiel: für $C_1 = 100\%$ sind mindestens zwei Testfälle nötig, z.B.
  - Pfad: A, B, C, D, E, B, F, G, H
  - Pfad: A, B, C, E, B, F, H
- Zweigtest ist stärkeres Kriterium als Anweisungstest, d.h. aus $C_1 = 100\%$ folgt auch $C_0 = 100\%$
- Nachteile
  - Verzweigungen: nur Ergebnis, keine Berücksichtigung komplexer Bedingungen
  - Schleifen werden eventuell nicht berücksichtigt
  - Sonderfälle (insbesondere Fehlerbehandlungen) können teilweise nur mit viel Aufwand simuliert werden



# Erinnerung: Aussagenlogik

- Aussagen
  - Aussage ist entweder wahr oder falsch (nie beides!), also vom Typ `boolean`
  - Elementaraussage (aka atomare Aussage), z.B.
    - `i > 0`
    - `x == y`
    - `value instanceof ArrayList`
  - Operator (aka Junktor) liefert neue Aussagen aus bestehenden Aussagen, z.B.
    - Negation (Nicht): `!value`
    - Konjunktion (Und): `first & second`
    - Disjunktion (Oder): `first | second`
    - Kontravalenz (exklusives Oder): `first ^ second`
- Hinweis: bedingte logische Operatoren && und ||
  - Aka Kurzschlussauswertung, Short-Circuit Evaluation
  - Zweiter Operand wird nicht ausgewertet, wenn Wahrheitswert bereits durch Auswertung des ersten Operanden feststeht: `false && second` bzw. `true || second`

# White-Box: Bedingungstest

- Bedingungstest (Branch Condition Testing)
  - Bestimme Testfälle, so dass jede atomare Aussage einer Bedingungsprüfung mindestens einmal mit `true` und mindestens einmal mit `false` getestet wird
  - Bei Bedingungen von Verzweigungen und Schleifen, aber auch bei Variablenzuweisungen
- Bedingungsüberdeckung (Branch Condition Coverage)
  - Anteil der mit `true` und `false` getesteten Aussagen von allen atomaren Aussagen in %
  - Für 100% sind mindestens $2$ und maximal $n + 1$ Testfälle nötig (mit Anzahl $n$ atomarer Aussagen; maximal $n + 1$ bei bedingten Operatoren)
- Beispiel: `if (x > 0 & y < 0) { _ }`
  - Für 100% genügen zwei Testfälle, z.B. (I) $x = 2, y = 2$ und (II) $x = -2, y = -2$
  - Bei && wären drei Fälle nötig, z.B. (I) $x = 1, y = 1$, (II) $x = 1, y = -1$ und (III) $x = 0, y = 0$
- Bedingungstest ist schwächeres Kriterium als Anweisungstest
  - Zwei Testfälle im Beispiel: $100\%$ Bedingungsüberdeckung, aber $C_0 < 100\%$, da If-Körper nie erreicht wird

# White-Box: Mehrfachbedingungstest

- Mehrfachbedingungstest (Branch Condition Combination Testing)
  - Bestimme Testfälle, so dass alle Kombinationen der Wahrheitswerte der atomaren Aussagen bei einer Bedingungsprüfung getestet werden (Verzweigungen / Schleifen)
  - Kombinatorische Explosion: für $n$ atomaren Aussagen sind das $2^n$ Testfälle!
- Mehrfachbedingungsüberdeckung wird entsprechend definiert
- Beispiel: `if (x > 0 & y < 0) { _ }`
  - Testfälle für 100%: (I) $x = 1, y = -1$, (II) $x = 2, y = 0$, (III) $x = 0, y = -2$ und (IV) $x = 0, y = 0$
- Beachte
  - Bei && maximal 50% Überdeckung, da zweite atomare Aussage nicht getestet wird, wenn $x \leq 0$
  - Wenn atomare Aussagen nicht unabhängig, dann sind eventuell nicht alle Kombinationen realisierbar, z.B. ist (`false`, `false`) nicht möglich für

```
if (x > 0 && x < 5) { _ }
```

- Mehrfachbedingungstest ist stärkeres Kriterium als Anweisungs- und Zweigtest

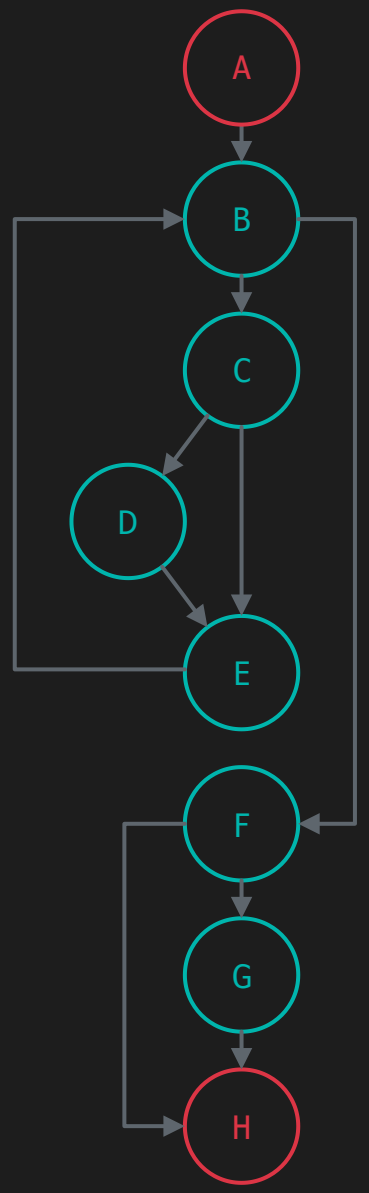
# White-Box: MC/DC

- Modified Condition/Decision Coverage (MC/DC)
  - Größere Aussagekraft als normaler Bedingungstest, aber ohne kombinatorische Explosion
  - Gefordert bei Luft- & Raumfahrt und Automobilindustrie für Komponenten mit hoher Kritikalität
- Tests müssen folgendes erfüllen
  - Jeder Ein- und Ausstiegspunkt wird aufgerufen
  - Jede Bedingungsprüfung nimmt jedes mögliche Ergebnis an [→ Zweigtest]
  - Jede atomare Aussage in einer Bedingungsprüfung nimmt Werte `true` und `false` an [→ Bedingungstest]
  - Für jede atomare Aussage einer Bedingungsprüfung gibt es Testfälle mit Ergebnis `true` bzw. `false`, so dass die isolierte Änderung dieser einen Aussage auch jeweils das Ergebnis ändert
- Beispiel: `if ((a | b) & c) { _ }`
  - Zwei Testfälle für vollständigen Zweig- & Bedingungstest: (I) alle `true`, (II) alle `false`
  - In beiden Testfällen ändert Wert für `b` das Ergebnis nicht, wenn `a` und `c` unverändert bleiben
  - Tatsächlich sind für MC/DC mindestens vier Testfälle nötig

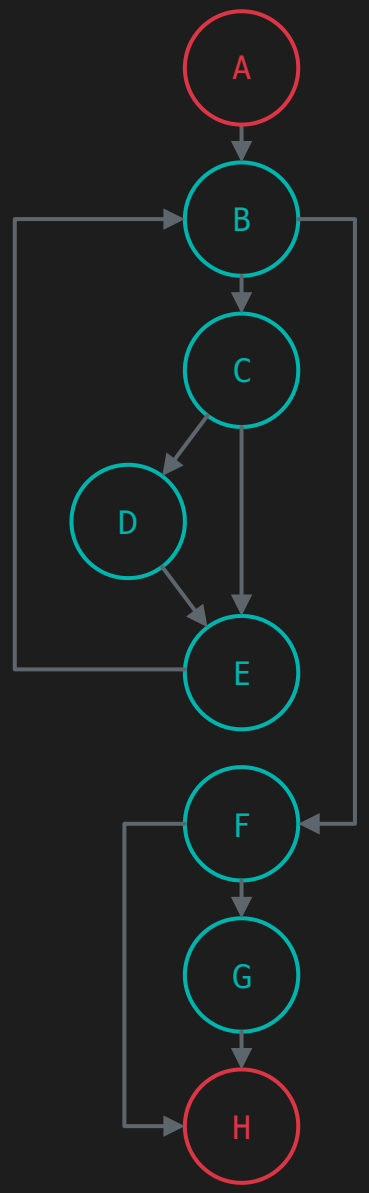
Quelle: (EUROCAE ED-12C:2012; NASA NPR 7150.2D:2022; ISO 26262-6:2018)

# White-Box: Pfadtest

- Pfadtest (Path Testing)
  - Bestimme Testfälle, so dass jeder Pfad mindestens einmal durchlaufen wird
- Pfadüberdeckung (Path Coverage)
  - Anteil durchlaufener Pfade von allen Pfaden in % (falls Beschränkung möglich)
- Beachte: durch Schleifen kombinatorische Explosion
  - Eher von theoretischem Interesse
  - Beschränkung ggf. durch fachliche Spezifikation oder technisch (z.B. Datentyp Integer)



# White-Box: Grenze-Inneres-Test

- Grenze-Inneres-Test (Boundary-Interior Testing)
  - Bestimme Testfälle, so dass jede Schleife erreicht wird und
    - gar nicht (bei kopfgesteuerter Schleife) und
    - genau einmal und
    - mehr als einmal

    vollständig durchlaufen wird (d.h. die Schleife wird nicht zwischendurch verlassen, z.B. durch `break`)
- Grenze-Inneres-Überdeckung (Boundary-Interior Coverage)
  - Anteil getesteter Schleifen von allen Schleifen in %
- Kriterium bezieht sich nur auf Schleifen
- Spezielle Tests für verschachtelte Schleifen
- Beispiel: für 100% sind mindestens drei Testfälle nötig
  - Pfad: A, B, F, H
  - Pfad: A, B, C, E, B, F, H
  - Pfad: A, B, C, E, B, C, E, B, F, H